
Tutorial de Desenvolvimento de um Jogo Arcade em Altera DE10

Sumário

1	Introdução	3
2	Base Teórica	3
2.1	Processador ARM Cortex-A9	3
2.2	A Placa DE10-Standard e o Sistema DE10-Standard Computer	4
2.3	Memória, Mapeamento e o Arquivo <code>address_map_arm.h</code>	5
2.3.1	Pontes FPGA (<i>FPGA Bridges</i>)	7
2.4	Componentes da FPGA	8
2.4.1	Componentes de Memória	8
2.4.2	Portas Paralelas	8
2.5	Porta JTAG	9
2.6	Saída de Vídeo VGA	9
2.6.1	Pixel Buffer	9
2.6.2	RGB Resampling	10
2.6.3	Double Buffering	10
2.6.4	Character Buffer	11
3	Guia de Instalação	12
3.1	Ferramentas Utilizadas	12
3.2	Instalação do Intel Quartus	13
3.3	Instalação do Altera Monitor Program	17
3.4	Configuração da placa DE10	18
3.5	Instalação do Windows Subsystem for Linux (WSL)	19
3.6	Primeira execução	20
3.6.1	Compilação utilizando o Intel FPGA Monitor Program	20
3.6.2	Compilação utilizando o script <i>compilar.bat</i>	23
4	Desenvolvimento do Jogo Bricks	25
4.1	Criação de Sprites	25
4.2	Objetivo do jogo	27
4.3	Organização geral do código	27
4.3.1	Funções de Vídeo Reaproveitadas do Exemplo	28
4.3.2	Funções de Desenho Adicionais	30
4.3.3	Estruturas e Constantes do Jogo	31
4.3.4	Montagem das Fases do Jogo	33
4.3.5	Sistema de Pontuação e Textos na Tela	34
4.3.6	Controle das Bolas	35
4.3.7	Sistema de Poderes	36
4.3.8	Funções Auxiliares	39
4.3.9	Lógica do Jogo	41

5	Execução do Projeto	50
6	Conclusão	51

1 Introdução

Este presente tutorial tem como objetivo guiar o leitor no desenvolvimento completo de um jogo do tipo *arcade* para a placa Altera DE10-Standard, utilizando o processador ARM Cortex-A9 e a linguagem C. O jogo desenvolvido é o *Bricks*, inspirado no clássico *Breakout*: uma barra controlada pelo jogador rebate uma ou mais bolas que destroem blocos posicionados na parte superior da tela, com sistema de pontuação, *power-ups* e condições de vitória e derrota.

Vale destacar que este documento aborda desde a instalação das ferramentas necessárias (Intel Quartus Prime e Altera Monitor Program) e a configuração inicial da placa, até o desenvolvimento detalhado de cada componente do jogo e a execução e verificação do funcionamento na placa física. Ao final deste tutorial, o leitor será capaz de compreender, desenvolver, compilar e executar um jogo funcional diretamente no hardware da DE10-Standard utilizando um processador ARM, com saída gráfica em monitor VGA e controle via botões físicos da placa.

2 Base Teórica

2.1 Processador ARM Cortex-A9

O ARM Cortex-A9 é um processador de 32 bits, cuja frequência é de 800 Mhz, pertencente à família Cortex-A da ARM, voltada para aplicações de alto desempenho como *smartphones* e *tablets*. Na placa DE10-Standard, ele está presente em configuração *dual-core*, que são dois núcleos idênticos implementados em hardware fixo dentro do chip Cyclone V SoC, cada um capaz de executar um programa de forma independente.

Um processador de 32 bits opera com palavras de dados de 32 bits por vez, o que define a largura dos seus registradores internos e a largura padrão das operações com a memória. Registradores são pequenas células de armazenamento internas ao processador, muito mais rápidas que a memória RAM, e é dentro deles que o processador realiza todos os seus cálculos.

Do ponto de vista de desempenho, o Cortex-A9 é um processador superescalar com execução fora de ordem, ou seja, ele despacha mais de uma instrução por ciclo de *clock* para diferentes unidades funcionais internas que operam em paralelo e o *pipeline* possui 8 estágios. Destaca-se que execução fora de ordem significa que a *CPU* pode reordenar dinamicamente a execução das instruções para evitar que o processador fique parado esperando um resultado, por exemplo se a instrução A depende de uma leitura da memória (lenta), o processador já adianta a execução da instrução B enquanto aguarda. Além disso, ele conta com previsão avançada de desvios (*advanced branch prediction*), ou seja, sempre que encontra uma instrução condicional (um

if, por exemplo), o processador antecipa qual caminho provavelmente será tomado e já começa a executá-lo antes mesmo de saber o resultado da condição, economizando bolhas no *pipeline*. Caso a previsão seja incorreta, as instruções executadas especulativamente são descartadas, o pipeline é esvaziado (*pipeline flush*) e a execução é reiniciada a partir do caminho correto. Embora isso introduza uma penalidade de alguns ciclos de clock, a elevada taxa de acerto do mecanismo de previsão faz com que o ganho médio de desempenho seja significativo.

O processador inclui também uma Unidade de Ponto Flutuante (FPU) com suporte a operações de precisão simples e dupla (adição, subtração, multiplicação, divisão, raiz quadrada) e uma *engine NEON*, que é um processador SIMD (*Single Instruction, Multiple Data*) de 128 bits que executa uma única instrução sobre múltiplos dados simultaneamente, útil para processamento de mídia e sinais [2].

Para o gerenciamento do acesso à memória com eficiência, o Cortex-A9 possui uma hierarquia de memórias *cache*: pequenas memórias internas extremamente rápidas que guardam automaticamente cópias dos dados e instruções acessados recentemente, evitando buscas repetidas na memória principal (muito mais lenta). Cada núcleo possui duas *caches* L1 privadas de 32 KB cada, sendo uma exclusivamente para instruções e outra para dados, e ambos compartilham uma *cache* L2 unificada de 512 KB.

O núcleo possui ainda uma MMU (*Memory Management Unit*), um circuito que traduz endereços virtuais (os que o programa enxerga) em endereços físicos reais. A MMU é necessária para rodar sistemas operacionais com memória virtual e isolamento de processos. Neste projeto, por não utilizarmos sistema operacional, o programa acessa diretamente os endereços físicos dos periféricos.

Cada núcleo possui ainda um conjunto de recursos adicionais para gerenciamento de tempo e eventos, como temporizadores privados e um controlador de interrupções compartilhado entre os dois núcleos. Esses recursos não são utilizados neste projeto, já que o controle de velocidade do jogo *Bricks* é feito por meio de um laço de espera simples em *software*, conforme será detalhado na Seção 4.3. Informações adicionais sobre os recursos apresentados e outros do processador estão presentes na documentação oficial [1].

2.2 A Placa DE10-Standard e o Sistema DE10-Standard Computer

A placa DE10-Standard possui como componente central o chip Cyclone V SoC da Intel/Altera. Esse chip reúne dois domínios distintos:

- o **Sistema de Processamento Físico** (*Hard Processor System* — HPS), que contém o processador ARM Cortex-A9 *dual-core*, uma porta de memória DDR3 e um conjunto de periféricos padrão (timers, GPIO, Ethernet, USB, entre outros);

- a **FPGA** (*Field-Programmable Gate Array*), uma matriz de lógica reconfigurável que pode ser programada para implementar circuitos digitais customizados. No caso deste presente tutorial, ela já vem pré-configurada com o sistema *DE10-Standard Computer*, que disponibiliza ao programador os controladores de vídeo, as portas paralelas dos botões, LEDs e *switches*, temporizadores de intervalo e outros periféricos.

Uma FPGA, diferentemente de um processador, não executa código: ela é o circuito digital. Cada periférico implementado nela (como o controlador VGA ou as portas de botões) é um bloco de hardware dedicado que opera de forma autônoma e paralela ao processador. A comunicação entre o processador ARM e esses periféricos acontece por meio das pontes FPGA (*FPGA Bridges*), que serão descritas na seção seguinte.

A Figura 1 apresenta o diagrama de blocos do sistema, que será a referência visual para toda a base teórica a seguir.

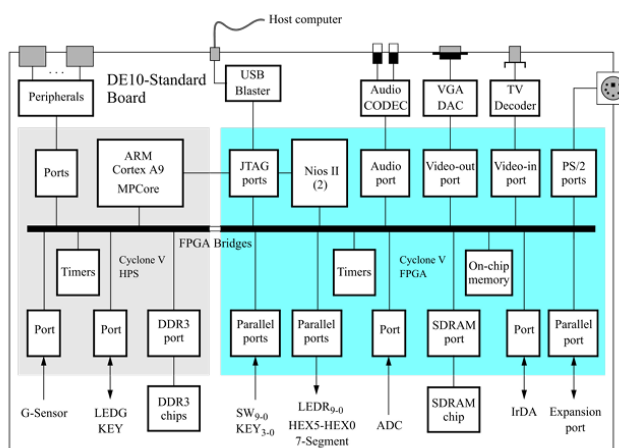


Figura 1: Diagrama de blocos do sistema DE10-Standard Computer (extraído de [1]).

2.3 Memória, Mapeamento e o Arquivo `address_map_arm.h`

O conceito fundamental para programar qualquer periférico nesta placa é o de **E/S mapeada em memória** (*memory-mapped I/O*). Para isso, utiliza-se um **barra-mento** (*bus*), que é um conjunto de sinais elétricos que interliga o processador a todos os demais componentes do sistema.

O barramento principal do sistema é o AMBA AXI (*Advanced Microcontroller Bus Architecture — Advanced eXtensible Interface*), um padrão aberto definido pela ARM. O AXI é responsável por conectar o processador ARM à memória DDR3, aos periféricos internos do HPS e, por meio das pontes FPGA descritas a seguir, a todos os periféricos presentes na FPGA. Em outras palavras, toda operação que o processador realiza ao executar uma instrução de leitura ou escrita em C trafega por esse barramento, fazendo com que cada periférico pareça, do ponto de vista do código, simplesmente uma posição de memória.

Cada periférico do sistema ocupa um intervalo fixo e exclusivo dentro do espaço de endereçamento do processador, definido e documentado pelo fabricante. Quando o processador coloca um endereço no barramento, a lógica de interconexão do sistema identifica a qual intervalo aquele endereço pertence e roteia a transação ao componente correspondente — seja a memória DDR3, um registrador de botões ou o controlador de vídeo na FPGA. Isso significa que acessar o estado dos botões ou escrever a cor de um pixel na tela é, do ponto de vista do código C, idêntico a ler ou escrever uma variável: basta declarar um ponteiro para o endereço do periférico por meio do *volatile* e operá-lo diretamente:

```
1 volatile int *KEY_ptr = (int *)0xFF200050;
2 int botoes = *KEY_ptr; // le o estado atual dos botoes KEY
```

Listing 1: Exemplo de acesso a periférico via memory-mapped I/O

O *volatile* informa ao compilador que ele nunca deve otimizar ou reordenar esse acesso, pois o valor no endereço pode mudar a qualquer momento por ação do hardware (um botão pode ser pressionado enquanto o processador executa outra instrução) [1]. Vale destacar que ao armazenar o valor lido em uma variável, essa variável passa a representar apenas uma cópia do estado do periférico naquele instante. Assim, em estruturas de repetição como *for* e *while*, é necessário realizar uma nova leitura do registrador a cada iteração. Caso contrário o programa continuará utilizando o valor obtido anteriormente, mesmo que o estado do hardware tenha sido alterado.

O arquivo `address_map_arm.h`, incluído no projeto é apenas um cabeçalho com constantes definidos por meio do `#define` que associam nomes legíveis aos endereços físicos dos periféricos. Com isso, é possível consultar o endereçamento e escrever diretamente no código.

```
1 #define KEY_BASE          0xFF200050
2 #define LED_BASE          0xFF200000
3 #define SW_BASE           0xFF200040
4 #define PIXEL_BUF_CTRL_BASE 0xFF203020
5 #define FPGA_CHAR_BASE    0xC9000000
6 #define RGB_RESAMPLER_BASE 0xFF203010
```

Listing 2: Trecho ilustrativo do arquivo `address_map_arm.h`

Portanto, ao escrever `(int *)KEY_BASE` no código, o compilador substitui `KEY_BASE` pelo endereço físico direto ao hardware `0xFF200050`. Já o mapa de memória do sistema, conforme documentado em [1], divide o espaço de endereçamento entre memória RAM e periféricos. Os endereços mais relevantes para este projeto são:

- `0x00000000` a `0x3FFFFFFF` — Memória DDR3 de 1 GB, organizada como $256\text{M} \times 32$ bits, onde o programa e os dados do jogo residem em execução;
- `0xFFFF0000` a `0xFFFFFFFF` — Memória interna de 64 KB de cada núcleo A9;

- 0xC0000000 a 0xC3FFFFFF — SDRAM externa de 64 MB;
- 0xC8000000 a 0xC803FFFF — Memória interna da FPGA de 256 KB, usada como *pixel buffer* de vídeo;
- 0xC9000000 a 0xC9001FFF — Memória interna da FPGA de 8 KB, usada como *character buffer* de vídeo;
- 0xFF200000 a 0xFF20000F — LEDs vermelhos;
- 0xFF200040 a 0xFF20004F — *Slider switches*;
- 0xFF200050 a 0xFF20005F — Botões de pressão KEY;
- 0xFF203020 a 0xFF20302F — Controlador do *pixel buffer*;
- 0xFF203030 a 0xFF203037 — Controlador do *character buffer*;
- 0xFF203010 — Módulo RGB Resampler.

2.3.1 Pontes FPGA (*FPGA Bridges*)

O barramento AXI do processador e o barramento interno da FPGA são dois domínios de hardware completamente independentes, cada um com seu próprio *clock* e sua própria estrutura de interconexão. Embora ambos estejam no mesmo chip, o processador ARM não consegue simplesmente colocar um endereço no barramento e esperar que a FPGA responda: é necessária uma ponte, que é um circuito dedicado que recebe transações de um barramento, as traduz para o protocolo do outro barramento e as retransmite, cruzando também a fronteira entre os dois domínios de *clock* de forma síncrona e controlada.

Existem três pontes no sistema:

- **HPS-to-FPGA:** a principal ponte de acesso do processador à FPGA. Possui largura de dados configurável (32, 64 ou 128 bits) e alta largura de banda.
- **Lightweight HPS-to-FPGA:** ponte de 32 bits, voltada especificamente para acesso a registradores de controle e status de periféricos na FPGA.
- **FPGA-to-HPS:** permite que lógica implementada na FPGA inicie acessos a recursos do HPS, como a memória DDR3. Não é utilizada diretamente neste projeto.

As pontes são habilitadas ou desabilitadas pelo registrador *Bridge reset* no endereço 0xFFD0501C: o bit 0 controla a ponte HPS-to-FPGA e o bit 1 controla a lightweight HPS-to-FPGA; valor 0 em cada bit significa ponte habilitada. Ao usar o *Intel FPGA Monitor Program* para carregar e executar o programa na placa, essa configuração é feita automaticamente antes do início da execução do código do usuário, logo não é necessário escrever código para isso.

2.4 Componentes da FPGA

2.4.1 Componentes de Memória

O sistema DE10-Standard Computer dispõe de três regiões de memória implementadas via FPGA ou controladores dedicados, todas acessíveis pelo processador ARM:

SDRAM: um controlador SDRAM na FPGA provê acesso a 64 MB de memória DRAM síncrona externa na placa ($32\text{M} \times 16$ bits), mapeada de `0xC0000000` a `0xC3FFFFFF`. Acessível por operações de palavra (32 bits), meia-palavra (16 bits) ou byte.

On-Chip Memory (Memória *Pixel Buffer*): 256 KB de memória implementada dentro da FPGA ($64\text{K} \times 32$ bits), ocupando os endereços `0xC8000000` a `0xC803FFFF`. É nessa região que o controlador de vídeo lê os dados de cor de cada pixel para gerar a imagem na tela. Esta é a memória central do sistema gráfico do jogo.

On-Chip Memory Character Buffer: 8 KB de memória implementada dentro da FPGA ($8\text{K} \times 8$ bits), ocupando `0xC9000000` a `0xC9001FFF`. É usada exclusivamente como buffer de caracteres para a porta de vídeo, conforme detalhado na Seção 2.6.

2.4.2 Portas Paralelas

As portas paralelas são a interface entre o processador e os dispositivos físicos de entrada/saída da placa. A estrutura geral de uma porta paralela consiste em até quatro registradores de 32 bits:

- **Data register** (endereço *Base*): nas portas de saída, o valor escrito aqui é aplicado diretamente nos pinos físicos; nas portas de entrada, a leitura aqui retorna o estado atual dos pinos;
- **Direction register** (*Base* + 4): presente em portas bidirecionais; bit em 0 = entrada, bit em 1 = saída;
- **Interruptmask register** (*Base* + 8): habilita geração de interrupções por porta;
- **Edgecapture register** (*Base* + C): registra transições de 0 para 1 nas entradas.

A porta mais relevante para este projeto é a dos botões de pressão KEY_{3-0} , com endereço base `0xFF200050`. O registrador utilizado no jogo é o **Data register** (`0xFF200050`), que é somente leitura e retorna 1 no bit correspondente enquanto o botão estiver pressionado e 0 quando solto. Os demais registradores dessa porta — **Interruptmask** (`0xFF200058`) e **Edgecapture** (`0xFF20005C`) — existem, mas não são utilizados neste projeto, já que o jogo verifica continuamente o estado atual dos botões a cada iteração do laço principal, ao invés de reagir a eventos de borda:

```

1 volatile int *KEY_ptr = (int *)KEY_BASE; // 0xFF200050
2 int botoes = *KEY_ptr;
3 if (botoes & 0x1) novo_x += VEL_PLAYER; // KEY0: move direita
4 if (botoes & 0x2) novo_x -= VEL_PLAYER; // KEY1: move esquerda

```

Listing 3: Leitura contínua dos botões no loop

2.5 Porta JTAG

A porta JTAG implementa o canal de comunicação serial entre a placa DE10 e o computador host. Ela é usada pelo *Intel FPGA Monitor Program* para carregar o programa compilado na placa e para exibir mensagens de texto no Terminal Window do Monitor Program. Ao executar o projeto, mensagens de *log* e de depuração enviadas pela placa aparecem nesse terminal graças à JTAG UART. Vale destacar que o usuário não precisa configurar a JTAG diretamente: o Monitor Program e o compilador cuidam disso automaticamente.

2.6 Saída de Vídeo VGA

A porta de vídeo do sistema DE10-Standard Computer é conectada ao controlador VGA embarcado na placa, permitindo ligar um monitor VGA padrão. Ela suporta resolução de tela de 640×480 pixels. A imagem gerada na tela é derivada de duas fontes independentes de memória: o *pixel buffer* e o *character buffer*.

2.6.1 Pixel Buffer

O *pixel buffer* é a região de memória que armazena a cor de cada pixel da imagem a ser exibida. Internamente, ele tem resolução de 320×240 pixels: como o monitor opera a 640×480 , o hardware replica cada valor de pixel tanto na dimensão x quanto na dimensão y ao gerar o sinal de vídeo, sem custo para o programador. Vale destacar que a coordenada $(0, 0)$ está no canto superior esquerdo da imagem.

Cada pixel é armazenado como uma variável de 16 bits no formato **RGB 5-6-5**: os 5 bits mais significativos representam a componente vermelha, os 6 bits seguintes representam o verde e os 5 bits menos significativos representam o azul. O endereço de um pixel na posição (x, y) é calculado combinando o endereço base do buffer com um deslocamento que codifica as coordenadas:

$$\text{endereço} = \text{base} + (y \times 1024) + (x \times 2)$$

O fator $\times 2$ existe porque cada pixel ocupa 2 bytes (16 bits). Por padrão, o endereço base é `0xC8000000` (início da On-Chip Memory da FPGA). Como exemplos concretos:

o pixel (0, 0) está em 0xC8000000; o pixel (1, 0) está em 0xC8000002; o pixel (0, 1) está em 0xC8000400; e o pixel (319, 239) está em 0xC803BE7E.

Já o fator $\times 1024$ decorre da estrutura interna do endereço: conforme apresentado na Figura 2, os 9 bits de x ocupam as posições de 1 até 9 do endereço e os 8 bits de y ocupam as posições de 10 até 17, o que equivale a deslocar y 10 bits para a esquerda, ou seja, multiplicá-lo por $2^{10} = 1024$.

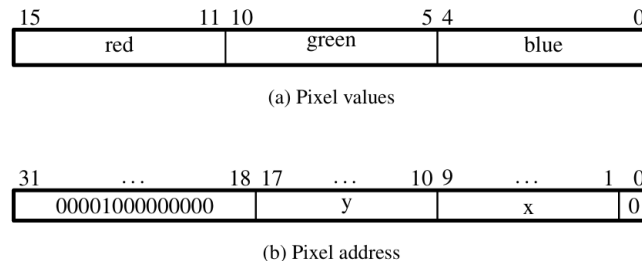


Figura 2: Valores e endereçamento do pixel (extraído de [1]).

2.6.2 RGB Resampling

O sistema DE10-Standard Computer possui um módulo de **RGB Resampling** (reamostrador de cor), cujo registrador de status está no endereço 0xFF203010. Esse módulo informa o formato de cor atual do sistema de vídeo. O formato padrão (modo 0x14) corresponde a 16 bits por pixel no formato RGB 5-6-5, descrito acima.

Na inicialização do programa, o código lê esse registrador para descobrir a profundidade de cor real do sistema e armazena o resultado na variável `db` (número de bits de cor). A partir daí, toda cor definida no código como um valor de 24 bits (formato 0xRRGGBB padrão) precisa ser convertida para o formato nativo antes de ser escrita no buffer, tal qual será detalhado na função `resample_rgb` na Seção 4.3.

2.6.3 Double Buffering

Um problema prático ao desenhar animações é o *flickering*: o hardware de vídeo lê o *pixel buffer* continuamente, quadro a quadro, para gerar o sinal VGA. Se o programa apagar e redesenhar os elementos do jogo enquanto o hardware está varrendo aquela região da memória, o usuário verá, por uma fração de segundo, uma imagem parcialmente desenhada, causando agitação visível na tela.

A solução documentada para esse problema é o **double buffering** [1]: o sistema mantém dois *pixel buffers* na memória.

- O **front buffer** é o buffer atualmente sendo lido pelo hardware de vídeo e exibido na tela. O programa não deve alterá-lo enquanto está sendo exibido.
- O **back buffer** é o buffer onde o programa desenha o próximo quadro do jogo, sem interferir na imagem atual.

O controlador de *pixel buffer* é o circuito dentro da FPGA responsável por gerenciar quais regiões de memória estão sendo usadas como *front* e *back buffer*. Ele expõe ao processador três registradores mapeados em memória, cada um com uma função específica:

- **Buffer register** (0xFF203020): armazena o *endereço* do *front buffer*, ou seja, o endereço da região de memória que o hardware de vídeo está atualmente lendo e exibindo na tela. Lendo esse registrador, o programa descobre onde está o buffer ativo. Escrever o valor 1 aqui não altera o endereço armazenado diretamente — é interpretado pelo controlador como um **comando de troca**: ao receber esse comando, o controlador agenda a troca entre os conteúdos do Buffer register e do Backbuffer register para o próximo sincronismo vertical.
- **Backbuffer register** (0xFF203024): armazena o *endereço* do *back buffer*, ou seja, a região de memória onde o programa deve desenhar o próximo quadro.
- **Status register** (0xFF20302C): o bit S vale 1 enquanto a troca ainda não foi concluída pelo hardware. O programa deve aguardar esse bit retornar a 0 antes de começar a desenhar o próximo quadro — caso contrário, correria o risco de desenhar no buffer errado.

Vale notar que, no projeto deste tutorial, as funções de desenho lêem o endereço base diretamente do Buffer register a cada chamada:

```
1 int pixel_buf_ptr = *(int *)PIXEL_BUF_CTRL_BASE; // lê o Buffer register
```

Listing 4: Leitura do endereço base do buffer ativo em video_box

Isso significa que, após cada troca de buffer realizada pelo hardware, as funções de desenho passam automaticamente a escrever no buffer correto, sem qualquer mudança no código de desenho. Diferentemente de outros projetos que alternam explicitamente entre *front* e *back buffer* a cada quadro, o jogo *Bricks* deste tutorial desenha diretamente sobre o único *pixel buffer* indicado pelo hardware, redesenhando apenas as pequenas regiões que efetivamente mudam de um quadro para o outro (a bola, o player e os elementos afetados), o que é suficiente para manter a fluidez da animação sem necessidade de alternância manual de buffers.

2.6.4 Character Buffer

O *character buffer* é uma segunda região de memória da porta de vídeo, usada para sobrepor texto à imagem do *pixel buffer*. Diferentemente do *pixel buffer*, onde o programador controla cada pixel individualmente, o *character buffer* opera com uma grade fixa de 80×60 células, em que cada célula corresponde a um bloco de 8×8 pixels na tela física. Essa grade não pode ser alterada: as 80 colunas e 60 linhas são definidas em hardware. O programador apenas decide *qual caractere* aparece em

cada célula, escrevendo seu código ASCII na posição de memória correspondente; o hardware converte automaticamente esse código para o padrão de pixels correto usando uma fonte de caracteres armazenada na memória da FPGA.

O endereço de um caractere é calculado a partir do endereço base `0xC9000000`, que corresponde ao caractere na posição $(0, 0)$. Cada linha tem 128 posições reservadas (mesmo que apenas 80 sejam usadas, pois o campo de x no endereço ocupa 7 bits, e $2^7 = 128$), então para chegar à linha y é necessário pular $y \times 128$ bytes; dentro da linha, soma-se x para alcançar a coluna desejada. O endereço final do caractere é:

$$\text{endereço} = 0xC9000000 + (y \times 128) + x$$

Para melhor exemplificação, destaca-se que a célula $(0, 0)$ está em `0xC9000000`; a célula $(1, 0)$ em `0xC9000001`; a célula $(0, 1)$ em `0xC9000080`; e a célula $(79, 59)$ em `0xC9001DCF`. No jogo *Bricks*, essa região é utilizada exclusivamente para exibir a pontuação e as mensagens de instrução do menu, como será visto na Seção 4.3.

Para aprofundamento em temporizadores, interrupções e demais recursos do sistema não cobertos neste tutorial, consulte o manual completo disponível em: http://fpgacademy.org/Downloads/DE10-Standard_Computer_ARM.pdf.

3 Guia de Instalação

3.1 Ferramentas Utilizadas

O desenvolvimento deste projeto requer um conjunto específico de ferramentas de hardware e software, descritas a seguir. Vale destacar que o processo de instalação e configuração de cada uma delas estão detalhados nas seções seguintes.

- **Intel Quartus Prime Lite:** é o ambiente de desenvolvimento integrado (IDE) da Intel para projetos com FPGAs, interligado em conjunto com o Altera Monitor Program para configuração e programação da placa. A versão Lite é gratuita e suficiente para os propósitos deste tutorial.
- **Altera Monitor Program:** é uma ferramenta também fornecida pela Intel, voltada especificamente para o desenvolvimento de programas em linguagem C ou Assembly para o processador ARM presente na placa. Ele integra em uma única interface o compilador, o carregador de programas e um terminal de depuração, permitindo compilar o código do jogo, transferi-lo para a placa via cabo USB-Blaster e acompanhar mensagens de *log* durante a execução.
- **Placa Altera DE10-Standard:** é o hardware do projeto, na qual o jogo é efetivamente executado. Ela deve estar conectada ao computador host por meio do

cabo USB-Blaster, que é o cabo responsável tanto pela programação da FPGA quanto pela comunicação com o Monitor Program durante o desenvolvimento.

- **Monitor VGA:** deve ser conectado ao conector VGA da placa para exibir a saída gráfica do jogo. Qualquer monitor com entrada VGA e suporte à resolução de 640×480 pixels é compatível.

O jogo é desenvolvido inteiramente em linguagem C, compilada pelo Monitor Program para o conjunto de instruções ARM. Destaca-se que não são utilizadas bibliotecas externas além das bibliotecas padrão `stdbool.h` e `stdlib.h` (esta última usada para alocação dinâmica de memória, como será visto na Seção 4.3), já que todo o acesso ao hardware é feito diretamente por ponteiros para os endereços dos periféricos, conforme descrito na seção anterior.

3.2 Instalação do Intel Quartus

As instruções a seguir descrevem o procedimento de instalação exatamente como foi realizado e verificado durante o desenvolvimento deste projeto. As versões de software e o sistema operacional Windows 11 indicados são os que garantidamente funcionam para reproduzir o ambiente de desenvolvimento descrito neste tutorial. Vale destacar que outras versões ou sistemas operacionais podem funcionar, mas estão fora do escopo deste documento e não foram validados pelos autores.

Para instalar o Intel Quartus Prime Lite versão 23.1 no Windows, acesse a página oficial de downloads em <https://www.altera.com/downloads/fpga-development-tools/quartus-prime-lite-edition-design-software-version-23-1-windows>. Em seguida, selecione as opções especificadas pela Figura 3.

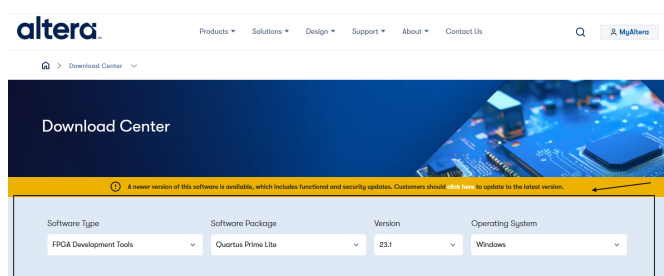


Figura 3: Instalação do Quartus

Role a página até a seção de *Installer* e clique na opção de *Multiple Download* e, logo embaixo, efetue o download, conforme indicado na Figura 4:

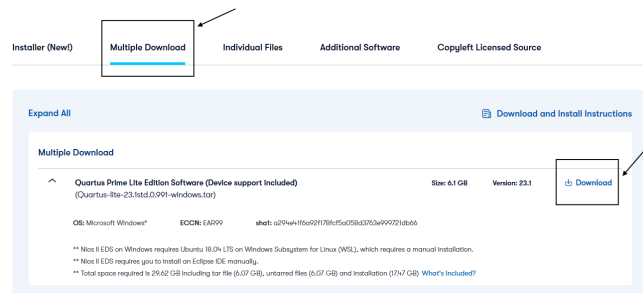


Figura 4: Realização de download

Após o download ser concluído, abra o Explorador de Arquivos do Windows (você pode fazer isso pressionando as teclas Windows + E simultaneamente) e clique na pasta Downloads no menu lateral esquerdo. Localize o arquivo baixado e selecione a opção Extrair Tudo... (*Extract All...*), clicando com o botão direito do mouse sobre o arquivo ou no botão Extrair Tudo (*Extract all*) localizado na barra superior do Explorador de Arquivos, conforme mostrado na imagem.

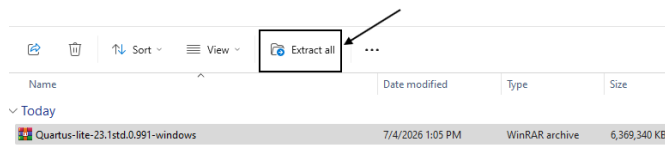


Figura 5: Extração do arquivo baixado

Após a extração ser concluída, localize a pasta criada e dê um duplo clique sobre ela para acessar os arquivos extraídos.

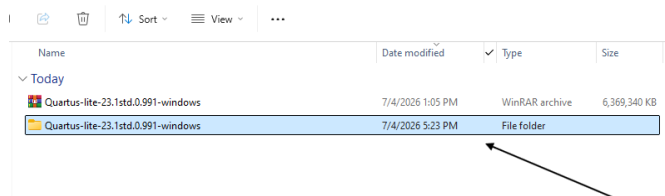


Figura 6: Arquivo extraído

Em seguida, localize o arquivo setup e dê um duplo clique sobre ele para iniciar a instalação. Uma importante observação durante a instalação é que o Windows poderá exibir mensagens de segurança, como avisos sobre o editor do programa, confirmação de permissões de administrador ou outras solicitações semelhantes. Sempre que isso ocorrer, clique em Sim, Executar (Run), Executar assim mesmo (Run anyway) ou na opção equivalente para prosseguir com a instalação.

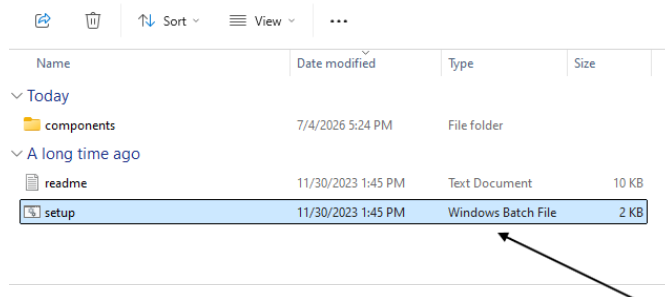


Figura 7: Execução do arquivo setup

Na janela de instalação, clique em *Next*, conforme mostrado na Figura 8. Em seguida, marque a opção *I accept the agreement*, conforme ilustrado na Figura 9, e clique novamente em *Next* para prosseguir com a instalação.

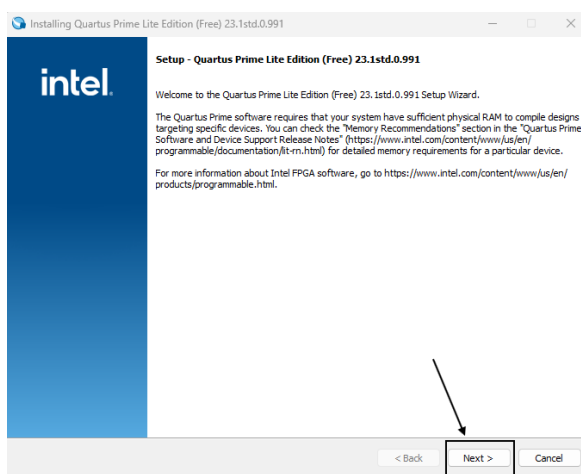


Figura 8: Execução do arquivo setup

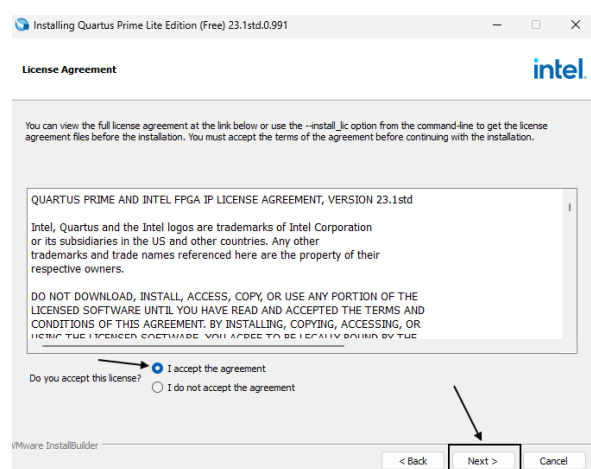


Figura 9: Aceitação dos termos

Na janela seguinte, será solicitado o diretório de instalação, conforme ilustrado na Figura 10. Recomendamos fortemente manter o diretório padrão sugerido (C:\intelFPGA_lite\23.1std), pois ele será referenciado nas etapas de configuração descritas nas seções seguintes. Após confirmar o diretório, clique em *Next* para prosseguir.

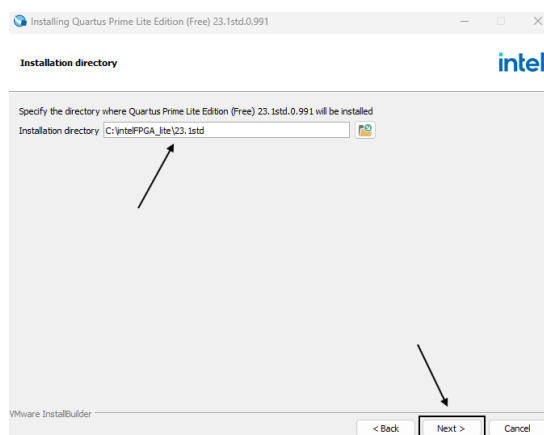


Figura 10: Seleção do diretório de instalação do Quartus

Em seguida, será exibida a tela de seleção de componentes. Marque as opções conforme indicado na Figura 11, selecionando os seguintes itens: Quartus Prime Lite Edition (Free), Quartus Prime (includes Nios II EDS), Quartus Prime Help, na seção *Devices*, marcar apenas Cyclone V, Ashling RiscFree IDE for Intel FPGAs, Questa - Intel FPGA Starter Edition e Questa - Intel FPGA Edition. Após selecionar os componentes, clique em *Next* para prosseguir.

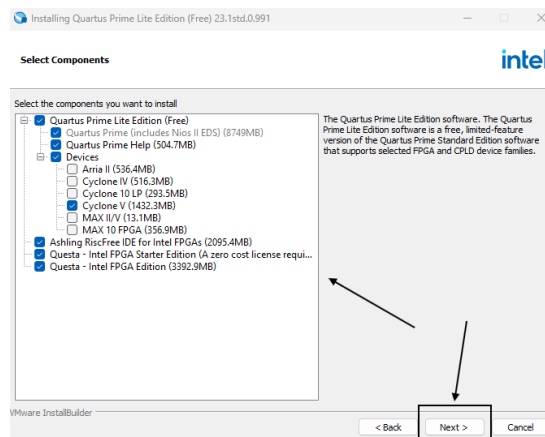


Figura 11: Seleção de componentes a ser instalado

Em seguida, será exibida uma tela de resumo com o espaço em disco necessário para a instalação. Não é necessário realizar nenhuma alteração nessa etapa, apenas clique em *Next*, conforme a Figura 12, para iniciar o processo de instalação.

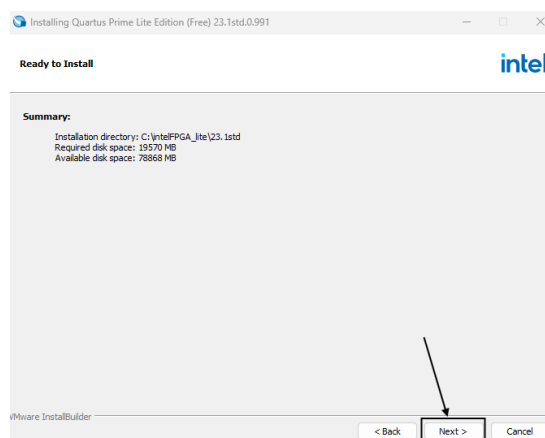


Figura 12: Tela de resumo

Ao final da instalação do Quartus, será exibida uma janela solicitando a instalação do driver do cabo USB-Blaster. Clique em *Launch USB Blaster device driver installation* e em seguida em *OK*. A tela de instalação do driver será aberta. Caso o driver já esteja instalado no sistema, ele será detectado automaticamente e basta clicar em *Finish* para concluir. Caso contrário, prossiga com a instalação normalmente, aceitando os termos e avançando até o final.

Com isso, a instalação do Intel Quartus Prime Lite está concluída. Embora o Quartus não seja utilizado diretamente durante o desenvolvimento do jogo, sua insta-

lação é necessária pois o Altera Monitor Program depende de componentes instalados junto a ele como o compilador ARM e as ferramentas de programação da FPGA para funcionar corretamente.

3.3 Instalação do Altera Monitor Program

Para a instalação do Altera Monitor Program, acesse <https://fpgacademy.org/tools> e role a página até encontrar a seção *Monitor Program*. Selecione a versão 21.1 e, em seguida, clique na opção *EXE* para iniciar o download do arquivo de instalação, conforme indicado na Figura 13.

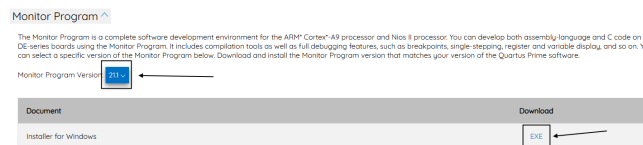


Figura 13: Download do Altera Monitor Program

Abra o Explorador de Arquivos do Windows pressionando *Windows + E*, navegue até a pasta *Downloads*, localize o arquivo baixado e dê dois cliques sobre ele para iniciar a instalação. Ao abrir o arquivo, será exibida a tela inicial de instalação do *Altera Monitor Program*, conforme ilustrado na Figura 14. Clique em *Next* para prosseguir. Na tela seguinte, conforme ilustrado na Figura 15, marque a opção *I agree* para aceitar os termos de uso e clique em *Next* para continuar.

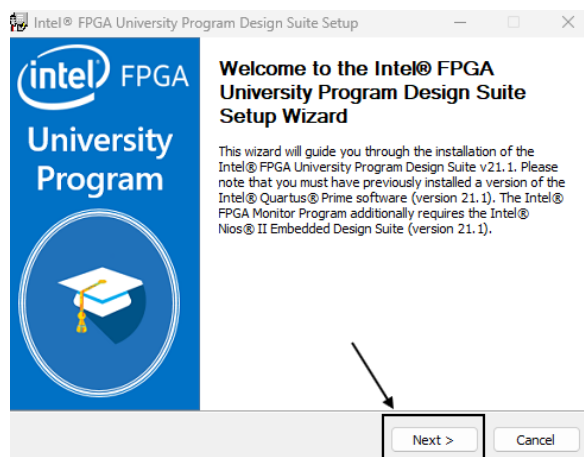


Figura 14: Tela inicial de instalação do Altera Monitor Program

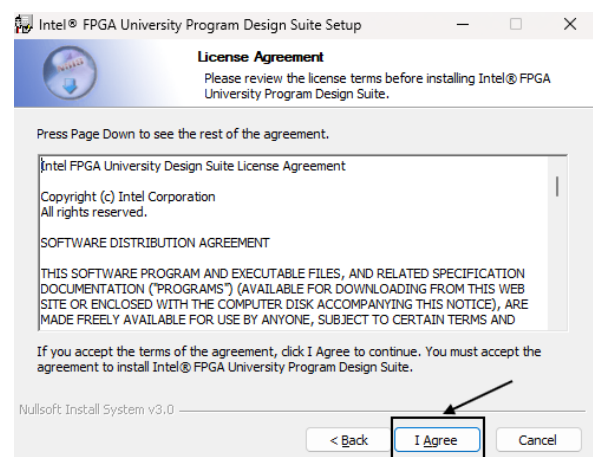


Figura 15: Aceitação dos termos de uso

Na etapa seguinte, será solicitado o caminho de instalação do Quartus, conforme ilustrado na Figura 16. Certifique-se de que o caminho informado seja:

C:\intelFPGA_lite\23.1std

que é exatamente o diretório utilizado durante a instalação do Quartus neste tutorial. Pode ser que o instalador exiba um aviso indicando que a versão recomendada é a 21.1. Mesmo com esse aviso, clique em *OK* para ignorar o aviso e prosseguir, pois a versão 23.1 é funcional com o *Altera Monitor Program* e foi validada neste tutorial.

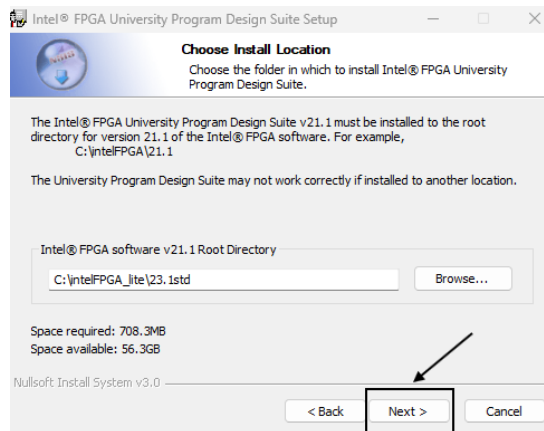


Figura 16: Configuração do caminho de instalação

Na etapa seguinte, será exibida a tela de seleção de componentes a instalar, conforme ilustrado na Figura 17. Recomendamos selecionar todos os componentes disponíveis ou, ao menos, o *DE10-Standard*, que é o sistema utilizado neste tutorial. Após realizar a seleção, clique em *Install* para iniciar a instalação.

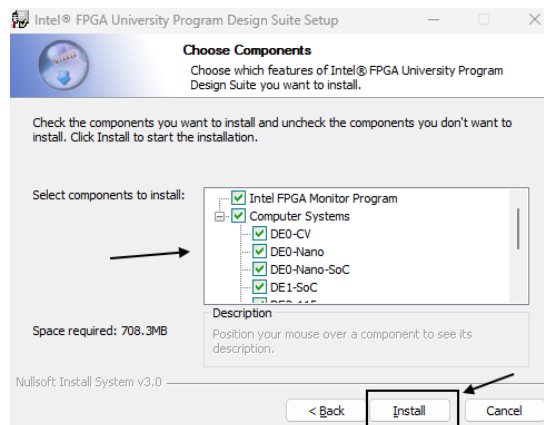


Figura 17: Seleção de componentes do *Altera Monitor Program*

Com isso, a instalação do *Altera Monitor Program* está concluída. Para abrir o programa, procure por *Intel FPGA Monitor Program* no menu inicial do Windows.

3.4 Configuração da placa DE10

Antes de executar qualquer programa na placa, é necessário realizar três conexões físicas. A primeira é a alimentação cuja entrada de energia própria está localizada na lateral da placa, onde deve ser conectado o adaptador de energia (fonte)

fornecido junto à placa e deve ser ligada a uma tomada. Sem essa conexão, a placa não ligará independentemente de quaisquer outros cabos conectados.

A segunda conexão é o cabo *USB-Blaster*: trata-se de um cabo USB comum, com conector USB-A em uma das extremidades (aquele retangular padrão, que vai no computador) e um conector USB-B na outra extremidade na FPGA (com formato semelhante a um quadrado). É por meio desse cabo que o computador se comunica com a placa para carregar programas.

A terceira conexão é o cabo VGA, responsável por levar o sinal de vídeo da placa ao monitor. Trata-se de um cabo com conectores trapezoidais de cor azul em ambas as extremidades. Conecte uma extremidade ao conector VGA da placa, identificado pela mesma cor azul e formato trapezoidal, e a outra extremidade à entrada VGA do monitor. Com as três conexões realizadas, ligue a placa pelo botão de energia localizado em sua superfície, na cor vermelha. Os LEDs da placa deverão acender, indicando que ela está energizada e pronta para uso.

3.5 Instalação do Windows Subsystem for Linux (WSL)

O *Windows Subsystem for Linux* (WSL) é um recurso do Windows que permite executar um ambiente Linux diretamente no sistema, sem necessidade de máquina virtual ou dual boot. Ele é necessário neste projeto pois o processo de compilação do código utiliza o comando *make*, que é uma ferramenta nativa do Linux e não está disponível nativamente no Windows.

Para instalar o WSL, abra o Prompt de Comando como administrador, ou seja, pressione a tecla *Windows*, digite `cmd`, clique com o botão direito sobre o *Prompt de Comando* e selecione *Executar como administrador*, conforme a Figura 18.

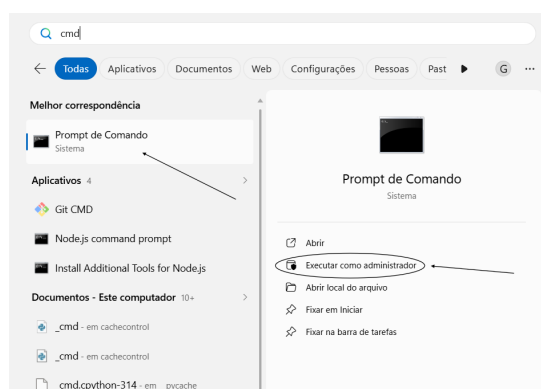


Figura 18: Abertura do Prompt de Comando

Na janela que se abrir, execute os seguintes comandos em sequência:

```
1 wsl --install
2 wsl --install -d Ubuntu
```

Listing 5: Instalação do WSL e do Ubuntu

O primeiro comando instala o WSL no sistema e o segundo instala o Ubuntu como distribuição Linux. Siga as instruções exibidas na tela. Será solicitado que você defina um nome de usuário e uma senha para o ambiente Linux. Com o ambiente configurado, execute os seguintes comandos no terminal do Ubuntu para atualizar os pacotes do sistema e instalar o *make*:

```
1 sudo apt update && sudo apt upgrade -y
2 sudo apt install make -y
```

Listing 6: Atualização dos pacotes e instalação do make

Com isso, o WSL está instalado e configurado com todas as dependências necessárias para o uso do programa descrito nas seções seguintes.

3.6 Primeira execução

Certifique-se de que esteja com todas as ferramentas instaladas e a placa devidamente conectada. Antes de prosseguir, é importante destacar que o Intel FPGA Monitor Program permite a compilação e o carregamento do programa diretamente por meio da opção *Compile & Load*. Entretanto, durante os testes realizados para este projeto, verificou-se que esse procedimento não funcionou corretamente na maioria dos computadores utilizados, embora tenha sido executado com sucesso em um dos ambientes de desenvolvimento.

A causa dessa diferença de comportamento não pôde ser determinada, podendo estar relacionada a diferenças na configuração do ambiente de software. Por esse motivo, este tutorial também apresenta um procedimento alternativo de compilação utilizando o script *compile.bat* e o carregamento manual do arquivo *.srec*, o qual foi validado com sucesso durante o desenvolvimento do projeto.

Nas subseções a seguir serão apresentados ambos os procedimentos de execução. Inicialmente, será descrito o fluxo padrão do Intel FPGA Monitor Program, utilizando a opção *Compile & Load*. Em seguida, será apresentado o procedimento alternativo baseado no script *compile.bat* e no carregamento manual do arquivo *.srec*, recomendado para os casos em que o método padrão não funcione corretamente.

3.6.1 Compilação utilizando o Intel FPGA Monitor Program

Abra o *Intel FPGA Monitor Program*. Em seguida, clique em *File* no menu superior e selecione *New Project*, conforme ilustrado na Figura 19.

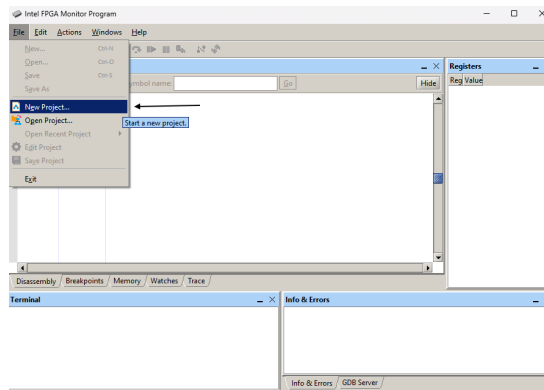


Figura 19: Criação de novo projeto no *Intel FPGA Monitor Program*

Na janela que se abrir, preencha o campo *Project directory* com o diretório onde deseja salvar o projeto e o campo *Project name* com o nome desejado. Em *Architecture*, selecione *ARM Cortex-A9*, conforme ilustrado na Figura 20. Em seguida, clique em *Next* para prosseguir. Logo após, na janela seguinte, em *Select a system*, selecione *DE10-Standard Computer*, conforme ilustrado na Figura 21, e clique em *Next* para prosseguir.

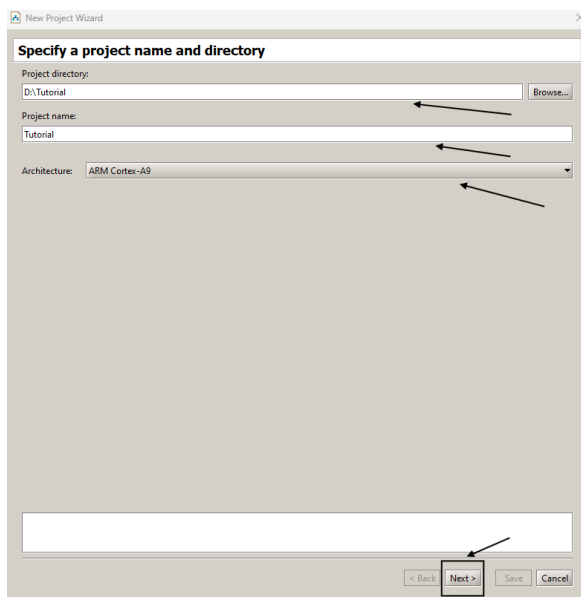


Figura 20: Configuração inicial do projeto

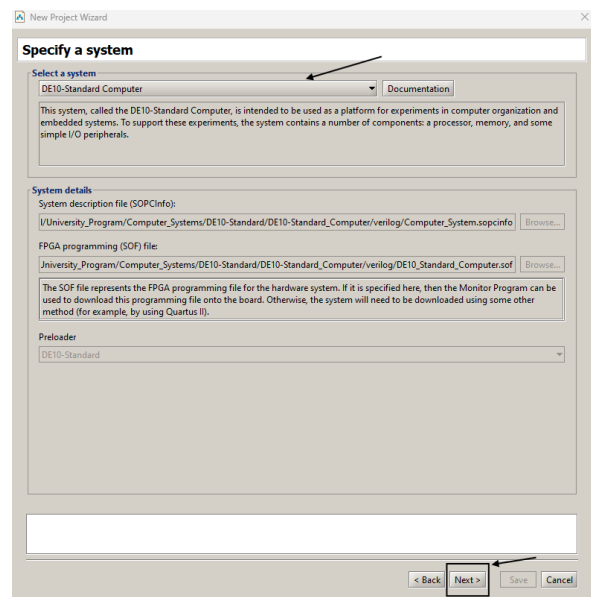


Figura 21: Seleção do sistema *DE10-Standard Computer*

Na janela seguinte, em *Program Type*, selecione *C Program*. Marque a opção *Include a sample program with the project* e, na lista de exemplos exibida, selecione *Video*, conforme ilustrado na Figura 22. Em seguida, clique em *Next* para prosseguir. Vale destacar que esse exemplo *Video* é o ponto de partida do código deste projeto: as funções de desenho na tela já vêm prontas nele, conforme será explicado em detalhes na Seção 4.3.

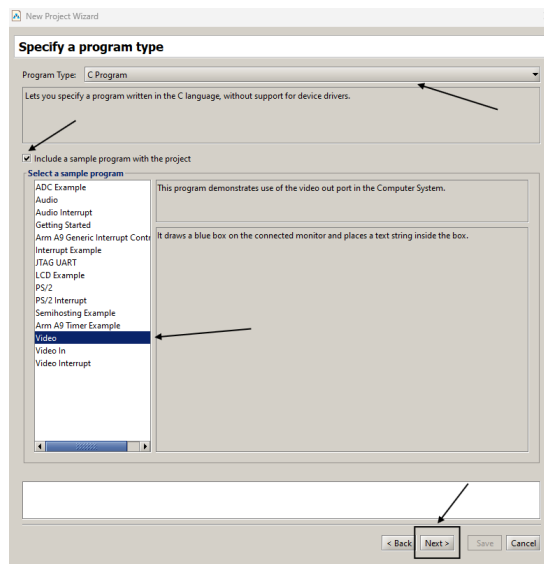


Figura 22: Seleção do tipo de programa e exemplo *Video*

Na janela *Specify Program Details*, são exibidos os arquivos que serão incluídos na compilação do projeto. Por padrão, os arquivos do exemplo selecionado anteriormente já estarão listados. Caso queira adicionar algum arquivo adicional, como os arquivos do próprio jogo desenvolvido neste tutorial, basta clicar em *Add* e localizar o arquivo desejado. Para remover um arquivo da lista, selecione-o e clique em *Remove*.

Na janela seguinte, *Specify System Parameters*, verifique se o campo *Host connection* exibe o nome do cabo USB-Blaster conectado. Caso o cabo não apareça, conforme ilustrado na Figura 24, certifique-se de que ele está devidamente conectado à placa e ao computador e clique em *Refresh* para atualizar a lista. Com o cabo detectado, clique em *Save* para finalizar a criação do projeto.

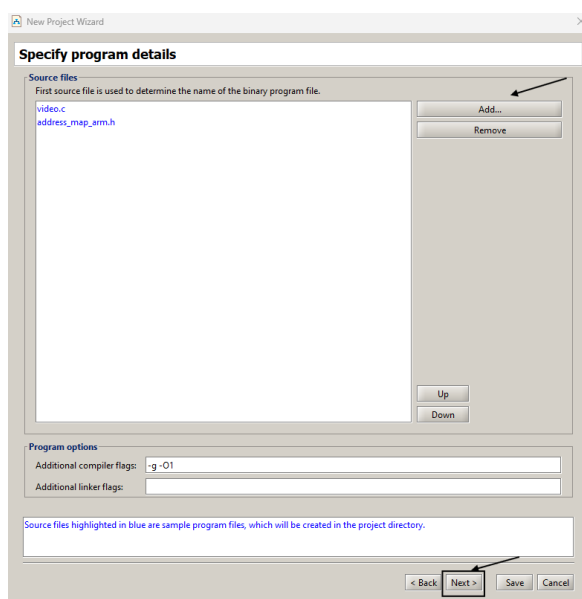


Figura 23: Gerenciamento de arquivos do projeto

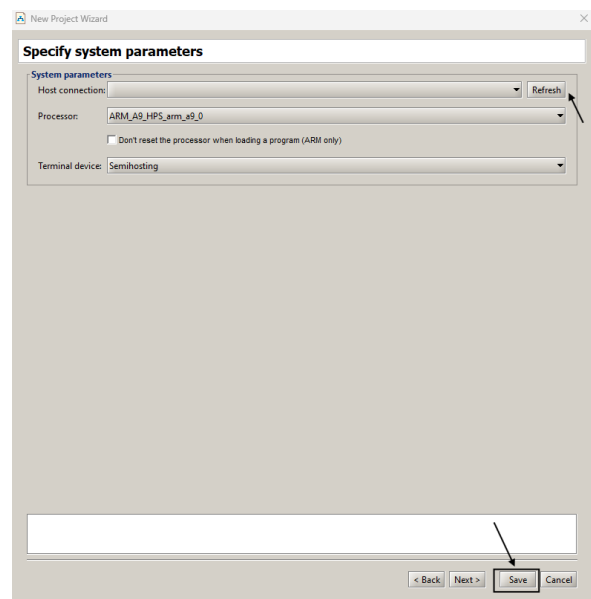


Figura 24: Verificação da conexão do cabo USB-Blaster

Ao finalizar essa etapa, será exibida uma janela de confirmação perguntando

se deseja realizar o download do sistema *DE10-Standard Computer* para a placa, conforme ilustrado na Figura 25. Clique em *Yes* para prosseguir. Destaca-se que deve garantir previamente que a placa esteja conectada ao computador pelo cabo correto e devidamente ligada, conforme o parágrafo anterior.

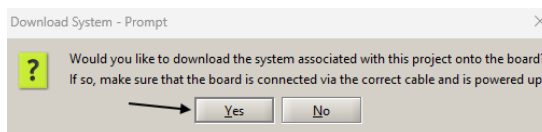


Figura 25: Download do sistema para a placa

Com o projeto criado, clique no menu *Actions* e selecione a opção *Compile & Load*, conforme ilustrado na Figura 26. Essa opção realiza a compilação do código e o carregamento do programa na placa.

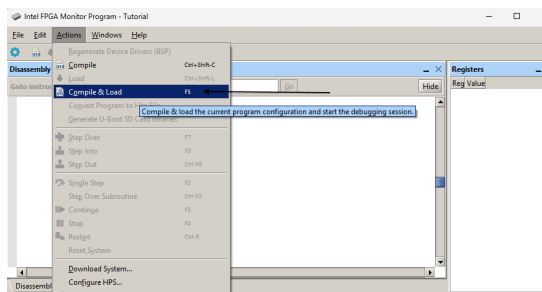


Figura 26: Compilação e carregamento do programa via *Compile & Load*

Em seguida, basta acessar novamente o menu *Actions* e clicar em *Continue* para que o programa seja de fato executado, ou simplesmente pressionar a tecla *F3*. O resultado da execução passará a ser exibido no monitor VGA conectado à placa. Esse é, portanto, o ciclo básico de utilização do *Intel FPGA Monitor Program* para todos os projetos desenvolvidos na FPGA: compilar e carregar o programa, e então executá-lo.

3.6.2 Compilação utilizando o script *compilar.bat*

Caso o fluxo de compilação e carregamento pelo *Intel FPGA Monitor Program* não funcione corretamente, uma alternativa é utilizar o *script* *compilar.bat*. Trata-se de um *script* que automatiza o processo de compilação do código em C, utilizando diretamente as ferramentas de compilação do *Altera Monitor Program*, localizadas dentro da pasta de instalação do Quartus. Destaca-se que por esse motivo, recomenda-se fortemente seguir os diretórios de arquivos padrão indicados durante a instalação das ferramentas, conforme descrito nas seções anteriores, garantindo que o *script* funcione corretamente caso seja necessário.

Para utilizá-lo, deve-se colocar o arquivo *compilar.bat* na mesma pasta do projeto, juntamente com os demais arquivos do código. Além disso, é necessário garantir

que o arquivo principal, ou seja, aquele que contém a função `int main()`, esteja nomeado exatamente como `main.c`, pois o *script* busca por esse nome especificamente dentro da pasta para realizar a compilação. A Figura 27 ilustra a pasta do projeto já com o arquivo `compilar.bat` posicionado junto aos demais arquivos. Feito isso, basta dar um duplo clique sobre o arquivo `compilar.bat` para que todo o processo de compilação seja executado automaticamente.

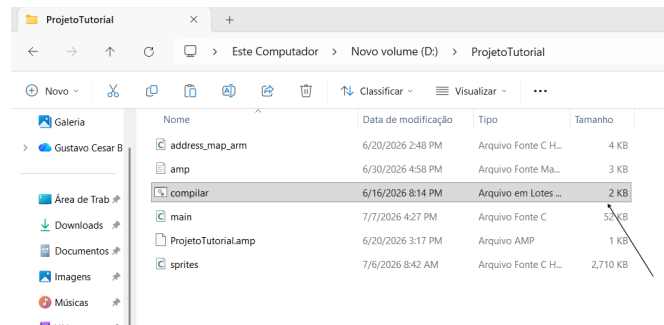


Figura 27: Pasta do projeto com o arquivo `compilar.bat`

Ao final da execução do *script*, é exibido no terminal o resultado da compilação e basta apenas pressionar qualquer tecla para fechar o prompt de comando, conforme ilustrado na Figura 28, sendo gerados tanto o arquivo `main.axf` quanto o arquivo `main.srec` dentro da mesma pasta do projeto. Recomenda-se utilizar o arquivo `main.srec` para o carregamento na placa.

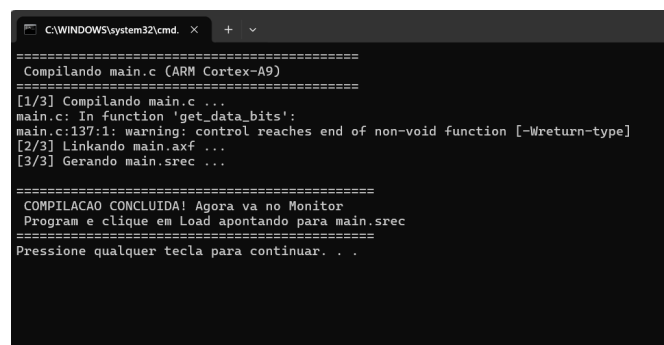


Figura 28: Resultado da compilação via `compilar.bat`

Com os arquivos gerados, deve-se criar um novo projeto no *Intel FPGA Monitor Program*, repetindo a mesma configuração inicial já descrita (arquitetura *ARM Cortex-A9* e sistema *DE10-Standard Computer*). Na etapa de *Program Type*, ao invés de *C Program*, deve-se selecionar a opção *AXF, ELF or SREC file*, conforme indicado na Figura 29.

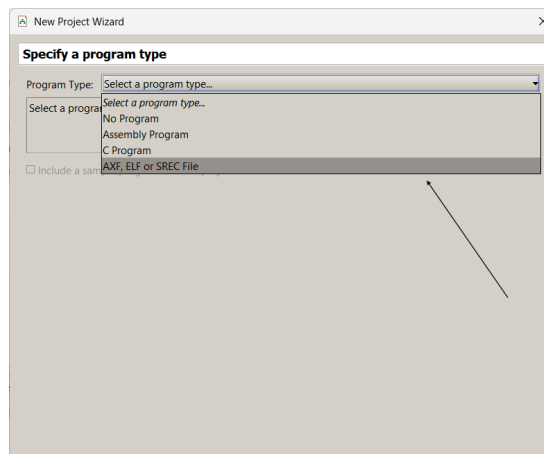


Figura 29: Seleção da opção AXF, ELF or SREC file

Em seguida, na tela de gerenciamento de arquivos do projeto (*Specify Program Details*), deve-se adicionar apenas o arquivo `main.srec` gerado pelo *script*, clicando em *Add* e localizando o arquivo dentro da pasta do projeto, conforme ilustrado na Figura 30.

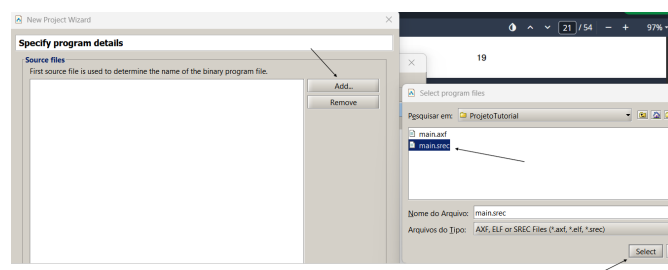


Figura 30: Adição do arquivo main.srec em Specify Program Details

A partir daqui, o restante das etapas segue exatamente o mesmo passo a passo já apresentado a partir da Figura 24 (verificação do cabo USB-Blaster em *Specify System Parameters* e download do sistema para a placa). A única diferença ocorre no momento da execução: ao invés de acessar o menu *Actions* e clicar em *Compile & Load*, deve-se clicar apenas em *Load*, acessando também o menu em *Actions*. Após o carregamento do programa ser concluído, basta clicar em *Continue*, ou pressionar *F3*, para executar o programa normalmente.

4 Desenvolvimento do Jogo Bricks

4.1 Criação de Sprites

Para a criação dos *sprites* utilizados no jogo, como os blocos, a bola e a barra, foi utilizado o site <https://www.piskelapp.com/>, uma ferramenta *online* voltada para a criação de *pixel art*. Ao acessar o site, basta clicar no botão *Create Sprite* para iniciar um novo desenho. Em seguida, é possível redimensionar o tamanho do

sprite desejado, em pixels de largura e altura, além de contar com diversas ferramentas de desenho, como pincel, balde de tinta e seleção de cores, conforme ilustrado na Figura 31. A partir dessas ferramentas, cada elemento gráfico do jogo pode ser desenhado pixel a pixel.

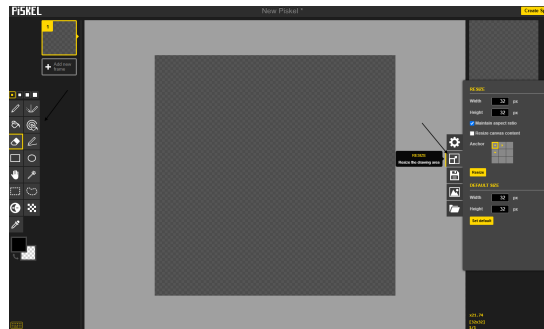


Figura 31: Interface do *Piskel*

Após finalizar o desenho do *sprite*, é necessário exportá-lo em um formato que possa ser utilizado diretamente no código em C do jogo. Para isso, clica-se na opção *Export*, em seguida em *Others* e, por fim, em *Download C file*, conforme ilustrado na Figura 32. Esse processo gera um arquivo em C contendo a representação do *sprite* na forma de uma matriz de cores.

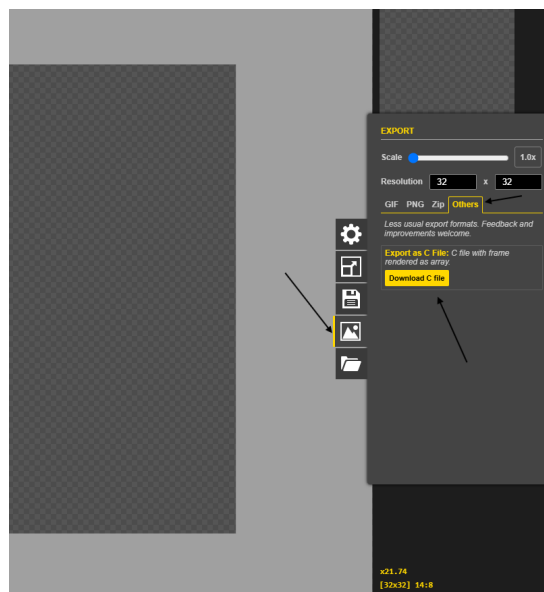


Figura 32: Exportação do arquivo em C no *Piskel*

Nesse arquivo gerado, o *sprite* é representado como uma matriz de inteiros, ocupando o cabeçalho `sprites.h` do projeto, em que cada elemento corresponde à cor de um único pixel do desenho, sendo representado como uma cor no formato de 24 bits, ou o valor -1 nas posições transparentes do desenho. A matriz possui, no total, uma quantidade de linhas e colunas igual à altura e à largura do *sprite* em pixels. De forma simplificada, o arquivo gerado segue uma estrutura semelhante à seguinte:

```

1 static const int sprite_poder[8][8] = {
2     {-1, -1, 0xFFD200, 0xFFD200, 0xFFD200, 0xFFD200, -1, -1},
3     {-1, 0xFFD200, 0xFFD200, 0xFFD200, 0xFFD200, 0xFFD200, 0xFFD200,
4         -1},
5     // ... outras linhas do sprite
6 };

```

Listing 7: Representação de um sprite no arquivo sprites.h

Logo, cada valor hexadecimal da matriz corresponde à cor de um pixel específico do *sprite*, sendo posteriormente convertido no código do jogo para o formato de cor nativo do *pixel buffer* da placa (RGB 5-6-5), descrito na Seção 2.6, antes de ser desenhado na tela, enquanto o valor -1 sinaliza ao código que aquele pixel deve ser ignorado, representando um fundo transparente.

4.2 Objetivo do jogo

O objetivo do *Bricks* consiste em o jogador controlar uma barra na parte inferior da tela, por meio dos botões KEY0 e KEY1, rebatendo a bola para destruir os blocos na parte superior e acumular pontos. O jogo possui duas fases com *layouts* diferentes de blocos, *power-ups* que podem dobrar o número de bolas e alterar temporariamente o tamanho da barra. Além disso, existem uma condição de vitória ao atingir a pontuação necessária em ambas as fases, isto é, destruir todos os blocos remanescentes nas duas fases, e uma condição de derrota caso todas as bolas caiam pela parte inferior da tela.

4.3 Organização geral do código

Antes de iniciar o desenvolvimento do código do jogo propriamente dito, o leitor deve partir do projeto de exemplo *Video*, utilizado anteriormente na Figura 22 durante a primeira execução no *Intel FPGA Monitor Program*. A escolha desse ponto de partida se justifica pelo fato do exemplo *Video* já trazer prontas algumas funções de acesso ao hardware gráfico da placa, como `video_text`, `video_box`, `resample_rgb` e `get_data_bits`, além do cabeçalho `address_map_arm.h`, que está devidamente configurado com todos os endereços dos periféricos da placa DE10-Standard. Dessa forma, o desenvolvimento deste projeto foi feito reaproveitando diretamente essas funções prontas do exemplo (mantendo-as praticamente inalteradas) e, em cima delas, adicionando uma quantidade consideravelmente maior de código com toda a lógica do jogo *Bricks*: estruturas de dados, novas funções de desenho, sistema de *power-ups*, e a máquina de estados que organiza o fluxo completo da partida.

Além do `address_map_arm.h`, o projeto conta também possui outro cabeçalho, o `sprites.h`, que armazena todos os *sprites* gráficos utilizados no jogo (tela de fundo

do menu, ícones dos quatro tipos de *power-up*, e as telas de *game over* e de vitória), no formato de matrizes de cores geradas pelo *Piskel*, conforme detalhado na Seção anterior. Assim, o projeto é organizado em apenas três arquivos: `main.c`, contendo o código reaproveitado do exemplo *Video* e toda a lógica do jogo e o cabeçalho `address_map_arm.h`, reaproveitado do exemplo.

A seguir, cada função do arquivo `main.c` será explicada em detalhes, começando pelas funções reaproveitadas do exemplo e pelas funções de desenho. Em seguida, serão explicadas as estruturas de dado, a lógica de cada função do jogo, e, por fim, a função `main` e sua máquina de estados.

4.3.1 Funções de Vídeo Reaproveitadas do Exemplo

As quatro funções descritas a seguir já vinham prontas no exemplo *Video* do *Intel FPGA Monitor Program* e foram mantidas praticamente sem alterações neste projeto, sendo apenas reaproveitadas em toda a lógica de desenho do jogo.

A função `video_text` escreve uma *string* de texto no *character buffer*, calculando o *offset* de cada caractere a partir da linha e coluna informadas, exatamente conforme a fórmula de endereçamento apresentada na Seção 2.6:

```
1 void video_text(int x, int y, char * text_ptr) {
2     int offset;
3     volatile char * character_buffer =
4         (char *)FPGA_CHAR_BASE; // video character buffer
5
6     /* assume that the text string fits on one line */
7     offset = (y << 7) + x;
8     while (*(text_ptr)) {
9         *(character_buffer + offset) =
10             *(text_ptr); // write to the character buffer
11         ++text_ptr;
12         ++offset;
13     }
14 }
```

Listing 8: Função `video_text`

Essa função é utilizada no jogo para exibir a pontuação atual e as mensagens de texto das telas de menu. Já a função `video_box` é a responsável por desenhar um retângulo sólido diretamente no *pixel buffer*, sendo a função de desenho mais utilizada em todo o jogo, já que é ela que desenha e apaga a barra do jogador, a bola, os blocos e as áreas de fundo. Antes de escrever qualquer pixel, a função lê o endereço do buffer ativo por meio do *Buffer register* (`PIXEL_BUF_CTRL_BASE`) e ajusta as coordenadas recebidas de acordo com a resolução de tela e o número de bits de cor (variáveis `res_offset` e `col_offset`), calculando o endereço de cada pixel dentro do retângulo com a mesma fórmula de endereçamento apresentada na Seção 2.6:

```

1 void video_box(int x1, int y1, int x2, int y2, short pixel_color) {
2     int pixel_buf_ptr = *(int *)PIXEL_BUF_CTRL_BASE;
3     int pixel_ptr, row, col;
4     int x_factor = 0x1 << (res_offset + col_offset);
5     int y_factor = 0x1 << (res_offset);
6     x1          = x1 / x_factor;
7     x2          = x2 / x_factor;
8     y1          = y1 / y_factor;
9     y2          = y2 / y_factor;
10
11     /* assume that the box coordinates are valid */
12     for (row = y1; row <= y2; row++) {
13         for (col = x1; col <= x2; ++col) {
14             pixel_ptr = pixel_buf_ptr +
15                 (row << (10 - res_offset - col_offset)) + (col
16                     << 1);
17             *(short *)pixel_ptr = pixel_color; // set pixel color
18         }
19     }
20 }

```

Listing 9: Função video_box

Por fim, as funções `resample_rgb` e `get_data_bits` cuidam da conversão de cor, sendo que a primeira converte uma cor de 24 bits (formato 0xRRGGBB) para o formato nativo de 16 ou 8 bits do *pixel buffer*, e a segunda traduz o modo retornado pelo módulo RGB Resampler (Seção 2.6) para a quantidade de bits de cor correspondente:

```

1 int resample_rgb(int num_bits, int color) {
2     if (num_bits == 8) {
3         color = (((color >> 16) & 0x000000E0) | ((color >> 11) & 0
4             x0000001C) |
5             ((color >> 6) & 0x00000003));
6         color = (color << 8) | color;
7     } else if (num_bits == 16) {
8         color = (((color >> 8) & 0x0000F800) | ((color >> 5) & 0
9             x000007E0) |
10             ((color >> 3) & 0x0000001F));
11     }
12     return color;
13 }

```

Listing 10: Funções resample_rgb e get_data_bits

A função `get_data_bits` segue a mesma lógica: um simples *switch-case* que mapeia o valor lido do registrador do RGB Resampler para a quantidade de bits de cor (1, 8, 9, 16, 24, 30, 32 ou 40 bits, dependendo do modo).

4.3.2 Funções de Desenho Adicionais

A partir das funções herdadas do exemplo, duas novas funções de desenho foram adicionadas para viabilizar o uso de *sprites* personalizados no jogo. A função `draw_pixel`, criada isolando a lógica interna de `video_box` para um único pixel, é a base sobre a qual as demais funções de *sprite* foram construídas:

```
1 void draw_pixel(int x, int y, short color) {
2     int pixel_buf_ptr = *(int *)PIXEL_BUF_CTRL_BASE;
3     int x_factor = 0x1 << (res_offset + col_offset);
4     int y_factor = 0x1 << res_offset;
5     x = x / x_factor;
6     y = y / y_factor;
7     int pixel_ptr = pixel_buf_ptr + (y << (10 - res_offset - col_offset)
8         ) + (x << 1);
9     *(short *)pixel_ptr = color;
10 }
```

Listing 11: Função `draw_pixel`

A partir de `draw_pixel`, a função `draw_background` percorre uma matriz de *sprite* do tamanho exato da tela (320x240), ignorando os pixels marcados como transparentes (valor -1) e convertendo cada pixel de 24 bits para o formato nativo antes de desenhá-lo:

```
1 void draw_background(int x, int y, int sprite[][320], int w, int h) {
2     int i = 0, j = 0;
3     for (i = 0; i < h; i++) {
4         for (j = 0; j < w; j++) {
5             if (sprite[i][j] != -1) {
6                 draw_pixel(x + j, y + i, resample_rgb(db, sprite[i][j]))
7             }
8         }
9     }
10 }
```

Listing 12: Função `draw_background`

Essa função é utilizada para desenhar as três telas de tela cheia do jogo: o menu inicial, a tela de *game over* e a tela de vitória. Já a função `draw_sprite` segue exatamente a mesma lógica, porém dimensionada para *sprites* pequenos de 8×8 pixels, utilizados nos ícones dos *power-ups*:

```
1 void draw_sprite(int x, int y, int sprite[][8], int w, int h) {
2     int i = 0, j = 0;
3     for (i = 0; i < h; i++) {
4         for (j = 0; j < w; j++) {
5             if (sprite[i][j] != -1) {
6                 draw_pixel(x + j, y + i, resample_rgb(db, sprite[i][j]))
7             }
8         }
9     }
10 }
```

```

7         }
8     }
9 }
10 }

```

Listing 13: Função draw_sprite

Nota-se que ambas as funções compartilham a mesma estrutura, diferindo apenas nas dimensões da matriz de *sprite* recebida como parâmetro, já que a linguagem C exige que o tamanho das colunas de uma matriz bidimensional seja conhecido em tempo de compilação.

4.3.3 Estruturas e Constantes do Jogo

Antes de descrever as funções de lógica do jogo, é necessário apresentar as estruturas e constantes que organizam o estado do *Bricks*. Os blocos são representados pela *struct* `Bloco`, guardando posição, cor, estado (vivo ou destruído) e o tipo de *power-up* que solta ao ser destruído:

```

1 typedef struct {
2     short Type;
3     short Pos_X;
4     short Pos_Y;
5     bool IsAlive;
6     short Contador;
7     int Cor;
8 } Bloco;
9
10 Bloco blocos[5][10];

```

Listing 14: Struct Bloco e matriz de blocos

A matriz `blocos` possui exatamente 5 linhas por 10 colunas, refletindo a grade máxima de blocos do jogo (usada integralmente na fase 1 e parcialmente na fase 2). O campo `Type` de cada bloco define qual *power-up* ele libera ao ser destruído, de acordo com as constantes:

```

1 #define BLOCO_TIPO_NORMAL 0
2 #define BLOCO_TIPO_DUPLICAR_BOLA 1
3 #define BLOCO_TIPO_AUMENTAR_PLAYER 2
4 #define BLOCO_TIPO_DIMINUIR_PLAYER 3
5 #define BLOCO_TIPO_RELOGIO 4

```

Listing 15: Constantes de tipo de bloco

Já as bolas são representadas pela *struct* `Bola`, contendo apenas a posição atual e o tipo de movimento (`Type_mov`), que define a direção da bola por meio de um valor de 0 a 5:

```

1 typedef struct {
2     short Type_mov;
3     short Pos_X;
4     short Pos_Y;
5 } Bola;
6
7 Bola *bolas = NULL;
8 int num_bolas = 0;

```

Listing 16: Struct Bola e array dinâmico

Destaca-se que o número de bolas em jogo pode variar dinamicamente, já que o jogo começa com uma única bola, mas o *power-up* de duplicação pode multiplicar esse número. Além disso, a *struct* Poder, que representa um *power-up* caindo na tela, também é organizada em um array dinâmico:

```

1 typedef struct {
2     short Pos_X;
3     short Pos_Y;
4     bool Ativo;
5     short Type; // 0=duplicar bola, 1=aumentar player, 2=diminuir player
6                 , 3=relogio
7 } Poder;
8
9 Poder *poderes = NULL;
10 int num_poderes = 0;

```

Listing 17: Struct Poder e array dinâmico

Por fim, o fluxo do jogo é organizado por meio de uma máquina de estados, representada pelo *enum* EstadoJogo, com cinco estados possíveis, e pelas constantes que definem a pontuação necessária para avançar de fase/estado:

```

1 typedef enum {
2     ESTADO_MENU ,
3     ESTADO_LEVEL1 ,
4     ESTADO_LEVEL2 ,
5     ESTADO_GAMEOVER ,
6     ESTADO_YOUWIN
7 } EstadoJogo;
8
9 #define SCORE_ALVO_LEVEL1 500
10 #define SCORE_ALVO_LEVEL2 300

```

Listing 18: Enum EstadoJogo e Constantes de Pontuação

4.3.4 Montagem das Fases do Jogo

A cor de cada bloco não é fixa: a função `AtualizarCor` deriva os componentes verde e azul da cor a partir da própria posição vertical do bloco (`Pos_Y`), fazendo com que os blocos de cada linha da grade recebam tonalidades ligeiramente diferentes, produzindo um efeito visual em degradê entre as fileiras:

```
1 int AtualizarCor(short Pos_Y) {
2     short G = Pos_Y + 20;
3     short B = Pos_Y + 30;
4     return ((Pos_Y & 0xff) << 16) + ((G & 0xff) << 8) + (B & 0xff);
5 }
```

Listing 19: Função `AtualizarCor`

A função `EstadoInicial` monta a matriz de blocos da primeira fase, preenchendo a grade de 5 linhas por 10 colunas por completo. Cada bloco recebe sua posição (calculada a partir do índice da linha e da coluna, com um espaçamento de 1 pixel entre eles), sua cor via `AtualizarCor`, é marcado como vivo, e desenhado na tela com `video_box`:

```
1 void EstadoInicial(){
2     short x = 10; short y = 10;
3     int i = 0, j = 0;
4     for (i = 0; i < 5; i++) {
5         for (j = 0; j < 10; j++) {
6             blocos[i][j].Type = 0;
7             blocos[i][j].Pos_X = x + (j * (LARGURA_BLOCO + 1));
8             blocos[i][j].Pos_Y = y + (i * (ALTURA_BLOCO + 1));
9             blocos[i][j].IsAlive = true;
10            blocos[i][j].Cor = AtualizarCor(blocos[i][j].Pos_Y);
11            video_box(blocos[i][j].Pos_X, blocos[i][j].Pos_Y,
12                    blocos[i][j].Pos_X + LARGURA_BLOCO - 1,
13                    blocos[i][j].Pos_Y + ALTURA_BLOCO - 1, blocos[i][j]
14                    ].Cor);
15        }
16    }
17    //Hardcore atribuicao dos tipos de blocos
18    blocos[2][4].Type = BLOCO_TIPO_DUPLICAR_BOLA;
19    blocos[2][5].Type = BLOCO_TIPO_AUMENTAR_PLAYER;
20    blocos[2][6].Type = BLOCO_TIPO_DIMINUIR_PLAYER;
21    blocos[2][7].Type = BLOCO_TIPO_RELOGIO;
22 }
```

Listing 20: Função `EstadoInicial`

Ao final da função, quatro blocos específicos são marcados com um dos tipos especiais de *power-up* via *hardcode*, garantindo que cada um dos quatro efeitos possíveis esteja sempre disponível em algum ponto da fase. A função `EstadoInicialLevel2` monta a matriz de blocos da segunda fase, reaproveitando a mesma grade de 5×10

posições, porém mantendo vivos apenas os blocos que formam um triângulo retângulo:

```
1 void EstadoInicialLevel2(){
2     short x = 10; short y = 10;
3     int i = 0, j = 0;
4     for (i = 0; i < 5; i++) {
5         short colunas_vivas_na_linha = 2 * (i + 1); // linha 0 -> 2,
6             linha 4 -> 10
7         for (j = 0; j < 10; j++) {
8             blocos[i][j].Type = 0;
9             blocos[i][j].Pos_X = x + (j * (LARGURA_BLOCO + 1));
10            blocos[i][j].Pos_Y = y + (i * (ALTURA_BLOCO + 1));
11            blocos[i][j].IsAlive = (j < colunas_vivas_na_linha);
12            blocos[i][j].Cor = AtualizarCor(blocos[i][j].Pos_Y);
13            if (blocos[i][j].IsAlive) {
14                video_box(blocos[i][j].Pos_X, blocos[i][j].Pos_Y,
15                    blocos[i][j].Pos_X + LARGURA_BLOCO - 1,
16                    blocos[i][j].Pos_Y + ALTURA_BLOCO - 1, blocos[
17                        i][j].Cor);
18            }
19        }
20    }
21    //Hardcore atribuicao dos tipos de blocos
22    if (blocos[2][4].IsAlive) blocos[2][4].Type =
23        BLOCO_TIPO_DUPLICAR_BOLA;
24    if (blocos[2][5].IsAlive) blocos[2][5].Type =
25        BLOCO_TIPO_AUMENTAR_PLAYER;
26    if (blocos[2][6].IsAlive) blocos[2][6].Type =
27        BLOCO_TIPO_DIMINUIR_PLAYER;
28    if (blocos[2][7].IsAlive) blocos[2][7].Type = BLOCO_TIPO_RELOGIO;
29 }
```

Listing 21: Função EstadoInicialLevel2

Nota-se que essa função é razoavelmente redundante em relação à EstadoInicial, repetindo boa parte de sua lógica, com o objetivo de isolar completamente a montagem da fase 2, permitindo o seu ajuste sem o risco de afetar a fase 1.

4.3.5 Sistema de Pontuação e Textos na Tela

A função AtualizarScore converte o valor inteiro da variável score para uma string de 4 dígitos, extraindo cada dígito por meio de operações de divisão e módulo, e a escreve no *character buffer* por meio da função video_text já apresentada:

```
1 void AtualizarScore(){
2     char texto[5];
3     texto[0] = '0' + (score / 1000) % 10;
4     texto[1] = '0' + (score / 100) % 10;
```

```

5     texto[2] = '0' + (score / 10)    % 10;
6     texto[3] = '0' + (score)        % 10;
7     texto[4] = '\0';
8     video_text(20, 55, texto);
9 }

```

Listing 22: Função AtualizarScore

Essa função é chamada sempre que um bloco é destruído, mantendo o placar exibido na tela sempre sincronizado com a variável `score`. Já a função `LimparTextoTela` apaga todo o texto residual das telas de menu, *game over* e vitória, sobrescrevendo as linhas correspondentes com espaços em branco, incluindo o texto do placar (que fica em uma camada de caracteres separada da tela de pixels e por isso não desaparece sozinho quando o fundo é redesenhado):

```

1 void LimparTextoTela(){
2     char text_top_row[40]      = "                \0";
3     char text_bottom_row[40]  = "                \0";
4     char text_score_clear[6]   = "        \0";
5     char text_pont_clear[5]    = "     \0";
6     video_text(35, 29, text_top_row);
7     video_text(32, 30, text_bottom_row);
8     video_text(5, 55, text_score_clear);
9     video_text(20, 55, text_pont_clear);
10 }

```

Listing 23: Função LimparTextoTela

4.3.6 Controle das Bolas

A função `inicializar_bolas` aloca dinamicamente o array `bolas` com `malloc`, começando sempre com uma única bola posicionada logo acima da barra do jogador, com movimento inicial nulo (`Type_mov = 0`, que corresponde a mover para baixo):

```

1 void inicializar_bolas() {
2     num_bolas = 1;
3     bolas = (Bola *)malloc(sizeof(Bola));
4     bolas[0].Pos_X = 155;
5     bolas[0].Pos_Y = Y_PLAYER - 15;
6     bolas[0].Type_mov = 0;
7 }

```

Listing 24: Função inicializar_bolas

Como o jogo representa a direção da bola por seis valores possíveis de `Type_mov` (0 = baixo, 1 = cima, 2 = diagonal superior-esquerda, 3 = diagonal superior-direita, 4 = diagonal inferior-esquerda, 5 = diagonal inferior-direita), a função `espelho_horizontal`

calcula o tipo de movimento espelhado horizontalmente de um valor informado, utilizado quando uma bola é duplicada, para que a nova cópia siga em uma direção diagonal oposta à original:

```
1 short espelho_horizontal(short tipo) {
2     if (tipo == 2) return 3;
3     if (tipo == 3) return 2;
4     if (tipo == 4) return 5;
5     if (tipo == 5) return 4;
6     return tipo; // 0 e 1 nao mudam (movimento vertical)
7 }
```

Listing 25: Função espelho_horizontal

Por fim, a função `duplicar_bolas` é utilizada quando o jogador coleta o *power-up* de duplicação de bola: ela redimensiona o array `bolas` com `realloc` para o dobro do tamanho atual, e para cada bola existente cria uma cópia na mesma posição, mas com o movimento espelhado horizontalmente:

```
1 void duplicar_bolas() {
2     bolas = (Bola *)realloc(bolas, sizeof(Bola) * num_bolas * 2);
3     int i = 0;
4     for (i = 0; i < num_bolas; i++) {
5         bolas[num_bolas + i].Pos_X = bolas[i].Pos_X;
6         bolas[num_bolas + i].Pos_Y = bolas[i].Pos_Y;
7         bolas[num_bolas + i].Type_mov = espelho_horizontal(bolas[i].
            Type_mov);
8     }
9     num_bolas *= 2;
10 }
```

Listing 26: Função duplicar_bolas

4.3.7 Sistema de Poderes

Quando um bloco especial é destruído, a função `adicionar_poder` cria um novo *power-up* caindo na posição do bloco destruído, redimensionando o array dinâmico `poderes` com `realloc`:

```
1 void adicionar_poder(short x, short y, short tipo) {
2     num_poderes++;
3     poderes = (Poder *)realloc(poderes, sizeof(Poder) * num_poderes);
4     poderes[num_poderes - 1].Pos_X = x;
5     poderes[num_poderes - 1].Pos_Y = y;
6     poderes[num_poderes - 1].Ativo = true;
7     poderes[num_poderes - 1].Type = tipo;
8 }
```

Listing 27: Função adicionar_poder

Como os *power-ups* são apagados da tela com uma chamada de `video_box` usando cor de fundo a cada frame, existe o risco de que um bloco ainda vivo posicionado por baixo daquela área fique com um pequeno "furo" causado por essa operação. A função `redesenhar_blocos` corrige esse problema, percorrendo todos os blocos vivos e redesenhando por completo qualquer um deles cuja área se sobreponha ao retângulo apagado:

```

1 void redesenhar_blocos(int x1, int y1, int x2, int y2, int db_local) {
2     int i = 0, j = 0;
3     for (i = 0; i < 5; i++) {
4         for (j = 0; j < 10; j++) {
5             if (!blocos[i][j].IsAlive) continue; //verifica se ta vivo
6             int bx1 = blocos[i][j].Pos_X; //pos x
7             int by1 = blocos[i][j].Pos_Y; // pos y
8             int bx2 = bx1 + LARGURA_BLOCO - 1; //pos x final do bloco
9             int by2 = by1 + ALTURA_BLOCO - 1; // pos y final do bloco
10            // Verifica sobreposicao entre o retangulo apagado e o bloco
11            if (x2 >= bx1 && x1 <= bx2 && y2 >= by1 && y1 <= by2) {
12                video_box(bx1, by1, bx2, by2, blocos[i][j].Cor);
13            }
14        }
15    }
16 }

```

Listing 28: Função `redesenhar_blocos`

A função `remover_poder` apaga um *power-up* específico da tela, e então compacta o array `poderes`, removendo o elemento indicado e redimensionando o array com `realloc` (ou liberando-o por completo com `free`, caso não sobre nenhum *power-up* ativo):

```

1 void remover_poder(int idx, short background_color) {
2     video_box(poderes[idx].Pos_X, poderes[idx].Pos_Y,
3             poderes[idx].Pos_X + 7, poderes[idx].Pos_Y + 7,
4             background_color);
5     //preenche com cor de fundo
6     int i = 0;
7     // compacta o array removendo o elemento idx
8     for (i = idx; i < num_poderes - 1; i++) {
9         poderes[i] = poderes[i + 1];
10    } //compacta
11    num_poderes--;
12    if (num_poderes > 0) {
13        poderes = (Poder *)realloc(poderes, sizeof(Poder) * num_poderes);
14    } else { //realoca e libera memoria
15        free(poderes);
16        poderes = NULL;
17    }
18 }

```

17 }

Listing 29: Função remover_poder

Por fim, a função atualizar_poderes é chamada a cada quadro do jogo e é responsável por todo o ciclo dos *power-ups* em queda, ou seja, apaga a posição antiga de cada um, avança sua posição vertical em 1 pixel, e trata os três possíveis desfechos. O primeiro trecho lida com a queda e a remoção de *power-ups* que saem da tela sem serem coletados:

```
1 void atualizar_poderes(short background_color) {
2     int i = 0;
3     for (i = num_poderes - 1; i >= 0; i--) {
4         video_box(poderes[i].Pos_X, poderes[i].Pos_Y,
5                 poderes[i].Pos_X + 7, poderes[i].Pos_Y + 7,
6                 background_color);
7         redesenhar_blocos(poderes[i].Pos_X, poderes[i].Pos_Y,
8                 poderes[i].Pos_X + 7, poderes[i].Pos_Y + 7,
9                 db);
10        //logica de desenho e redesenho
11        poderes[i].Pos_Y += 1;
12        //lida com a queda
13        //Se passar o player entao e removido
14        if (poderes[i].Pos_Y > 210) {
15            remover_poder(i, background_color);
16            continue;
17        }
18    }
```

Listing 30: Função atualizar_poderes

A segunda parte da função verifica a colisão do *power-up* com a barra do jogador e aplica o efeito correspondente ao seu Type:

```
1     if ((poderes[i].Pos_Y + 7 >= Y_PLAYER) &&
2         (poderes[i].Pos_Y <= Y_PLAYER + ALTURA_PLAYER) &&
3         (poderes[i].Pos_X + 7 >= player_x) &&
4         (poderes[i].Pos_X <= player_x + LARGURA_PLAYER)) {
5         if (poderes[i].Type == 0) duplicar_bolas(); //caso duplicar bola
6         if (poderes[i].Type == 1) {
7             LARGURA_PLAYER = LARGURA_PLAYER_EXPANDIDA; //caso largura
8             aumentar
9             contador_expansao = DURACAO_EXPANSAO;
10            tipo_efeito_player = 1;
11        }
12        if (poderes[i].Type == 2) {
13            LARGURA_PLAYER = LARGURA_PLAYER_REDUZIDA; //caso largura
14            reduzir
15            contador_expansao = DURACAO_EXPANSAO;
16            tipo_efeito_player = 2;
17        }
18    }
```

```

16         if (poderes[i].Type == 3) {
17             contador_relogio = DURACAO_RELOGIO; //caso tempo lento
18         }
19         remover_poder(i, background_color);
20         continue;
21     }

```

Listing 31: Função atualizar_poderes

Nota-se que os efeitos de aumento e redução de largura compartilham a mesma variável de controle `contador_expansao` e a mesma variável `tipo_efeito_player`, já que apenas um desses dois efeitos pode estar ativo por vez (o mais recente sempre substitui o anterior), enquanto o efeito de relógio possui seu próprio contador independente (`contador_relogio`), podendo estar ativo simultaneamente a qualquer um dos outros efeitos. Por fim, caso o *power-up* não tenha saído da tela nem colidido com o jogador, a última parte da função apenas redesenha seu *sprite* na nova posição, de acordo com seu tipo:

```

1     if (poderes[i].Type == 0) draw_sprite(poderes[i].Pos_X, poderes[i].
        Pos_Y, sprite_poder, 8, 8); //duplica bola
2     if (poderes[i].Type == 1) draw_sprite(poderes[i].Pos_X, poderes[i].
        Pos_Y, sprite_poder_largura, 8, 8); //largura aumenta
3     if (poderes[i].Type == 2) draw_sprite(poderes[i].Pos_X, poderes[i].
        Pos_Y, sprite_poder_reduzir, 8, 8); //largura diminui
4     if (poderes[i].Type == 3) draw_sprite(poderes[i].Pos_X, poderes[i].
        Pos_Y, sprite_poder_relogio, 8, 8); //relogio
5     }
6 }

```

Listing 32: Função atualizar_poderes

4.3.8 Funções Auxiliares

A função `aguardar_soltar_tecla` bloqueia a execução em um laço vazio enquanto o botão indicado pela máscara de bits continuar pressionado, e foi criada especificamente para fazer o reset do jogo (botão KEY2) funcionar corretamente: sem ela, o mesmo pressionamento de KEY2 que trouxe o jogador de volta ao ESTADO_MENU continuaria sendo lido como "pressionado" assim que o laço da tela de menu começasse, fazendo o jogo pular a tela inicial e iniciar uma nova partida instantaneamente. Por isso, toda vez que KEY2 é detectado, o código chama essa função para travar ali até o jogador soltar o botão, só então prosseguindo para o próximo estado:

```

1 void aguardar_soltar_tecla(volatile int *KEY_ptr, int mascara) {
2     while (*KEY_ptr & mascara);
3 }

```

Listing 33: Função aguardar_soltar_tecla

A função `TelaInicial` desenha o fundo do menu (*sprite* palmeiras) por meio de `draw_background`, exibe o título e a instrução do jogo via `video_text`, e permanece em um laço aguardando que o jogador pressione `KEY0` para iniciar a partida, ou `KEY2` para sair do menu:

```
1 void TelaInicial(){
2     volatile int *KEY_ptr = (int *)KEY_BASE;
3     draw_background(0, 0, palmeiras, 320, 240);
4     char text_top_row[40]    = "Jogo_breakout\0";
5     char text_bottom_row[40] = "Aperte_Key0_para_start\0";
6     video_text(35, 29, text_top_row);
7     video_text(32, 30, text_bottom_row);
8     while (1) {
9         int botoes = *KEY_ptr;
10        if (botoes & 0x1) return; // KEY0
11        if (botoes & 0x4) {
12            aguardar_soltar_tecla(KEY_ptr, 0x4);
13            return;
14        }
15    }
16 }
```

Listing 34: Função `TelaInicial`

Por fim, a função `Reset` centraliza toda a preparação necessária antes do início de uma fase (seja a primeira vez que o jogador entra na fase 1, seja a transição para a fase 2). Primeiramente, zera-se a pontuação e atualiza-se o placar, redesenha o fundo da área de jogo, reposiciona a barra do jogador no centro com a largura padrão, libera qualquer bola ou *power-up* remanescente da partida anterior, e monta a matriz de blocos correspondente à fase atual:

```
1 void Reset(EstadoJogo fase){
2     score = 0;
3     AtualizarScore();
4     video_box(10, 10, 309, 209, resample_rgb(db, BACKGROUND_BLUE));
5     //reajusta posicionamento player
6     player_x = 135;
7     LARGURA_PLAYER = LARGURA_PLAYER_BASE;
8     contador_expansao = 0;
9     contador_relogio = 0;
10    //libera dinamicamente arrays
11    if (bolas) free(bolas);
12    bolas = NULL;
13    if (poderes) free(poderes);
14    poderes = NULL;
15    num_poderes = 0;
16    //Inicializa uma fase
17    if (fase == ESTADO_LEVEL2) EstadoInicialLevel2();
18    else EstadoInicial();
19 }
```

```

19     inicializar_bolas();
20     //Inicializa o player
21     video_box(player_x, Y_PLAYER, player_x + LARGURA_PLAYER - 1,
22               Y_PLAYER + ALTURA_PLAYER - 1, resample_rgb(db, COR_PLAYER)
23               );
24 }

```

Listing 35: Função Reset

Note que a função `Reset` zera incondicionalmente a variável `score`; é justamente por esse motivo que, na transição da fase 1 para a fase 2 (descrita a seguir na máquina de estados), a pontuação acumulada precisa ser salva em uma variável temporária antes da chamada e restaurada logo em seguida, garantindo que o jogador não perca o progresso obtido na primeira fase.

4.3.9 Lógica do Jogo

A função `main` organiza a execução do jogo por meio de uma máquina de estados com cinco estados possíveis: `ESTADO_MENU`, `ESTADO_LEVEL1`, `ESTADO_LEVEL2`, `ESTADO_GAMEOVER` e `ESTADO_YOUWIN`. No início da função, são reaproveitados os mesmos trechos de inicialização de vídeo já presentes no exemplo *Video*, seguidos da declaração da variável `estado`, iniciada em `ESTADO_MENU`:

```

1  int main(void) {
2      volatile int * video_resolution = (int *) (PIXEL_BUF_CTRL_BASE + 0x8);
3      screen_x = *video_resolution & 0xFFFF;
4      screen_y = (*video_resolution >> 16) & 0xFFFF;
5
6      volatile int * rgb_status = (int *) (RGB_RESAMPLER_BASE);
7      db = get_data_bits(*rgb_status & 0x3F);
8
9      res_offset = (screen_x == 160) ? 1 : 0;
10     col_offset = (db == 8) ? 1 : 0;
11
12     volatile int *KEY_ptr = (int *)KEY_BASE;
13     short background_color = resample_rgb(db, BACKGROUND_BLUE);
14
15     EstadoJogo estado = ESTADO_MENU;
16
17     while (1) {
18         switch (estado) {
19             case ESTADO_MENU: {
20                 TelaInicial();
21                 LimparTextoTela();
22                 estado = ESTADO_LEVEL1;
23                 break;
24             }

```

Listing 36: Função main

No estado ESTADO_MENU, a função TelaInicial é chamada e, assim que o jogador pressiona um botão, o texto é apagado e o jogo avança para ESTADO_LEVEL1. O trecho a seguir mostra o início do estado ESTADO_LEVEL1, no qual a fase é montada por meio da função Reset:

```
1 case ESTADO_LEVEL1: {
2     video_text(5, 55, text_score);
3     video_text(20, 55, text_pontoacao);
4     video_box(0, 0, STANDARD_X - 1, STANDARD_Y - 1, 0); // fundo preto
5     video_box(10, 10, 310 - 1, 210 - 1, background_color);
6
7     Reset(ESTADO_LEVEL1);
8
9     bool fase_ativa = true;
10    while (fase_ativa) {
11        int botoes = *KEY_ptr;
12        short novo_x = player_x;
13
14        if (botoes & 0x1) novo_x += VEL_PLAYER; // KEY0: direita
15        if (botoes & 0x2) novo_x -= VEL_PLAYER; // KEY1: esquerda
16
17        if (botoes & 0x04) { // KEY2: reset volta para o menu
18            aguardar_soltar_tecla(KEY_ptr, 0x04);
19            LimparTextoTela();
20            estado = ESTADO_MENU;
21            fase_ativa = false;
22            break;
23        }
```

Listing 37: Função main - início do estado LEVEL1

A cada iteração do laço principal da fase, o estado dos botões é lido uma única vez, calculando a posição horizontal desejada para a barra (`novo_x`), e o botão KEY2 é verificado para permitir que o jogador reinicie o jogo a qualquer momento, retornando ao menu. O trecho seguinte trata da movimentação da barra, redesenhando apenas a fatia de pixels que efetivamente muda de posição, ao invés de apagar e redesenhar a barra inteira a cada quadro:

```
1 if (novo_x < 10) novo_x = 10;
2 if (novo_x > 310 - LARGURA_PLAYER) novo_x = 310 - LARGURA_PLAYER;
3
4 if (novo_x != player_x) {
5     if (novo_x > player_x) {
6         video_box(player_x, Y_PLAYER, novo_x - 1,
7             Y_PLAYER + ALTURA_PLAYER - 1, resample_rgb(db,
8                 BACKGROUND_BLUE));
9     } else {
10        video_box(novo_x + LARGURA_PLAYER, Y_PLAYER,
11            player_x + LARGURA_PLAYER - 1, Y_PLAYER +
```

```

11             ALTURA_PLAYER - 1,
               resample_rgb(db, BACKGROUND_BLUE));
12     }
13     player_x = novo_x;
14     video_box(player_x, Y_PLAYER, player_x + LARGURA_PLAYER - 1,
15             Y_PLAYER + ALTURA_PLAYER - 1, resample_rgb(db, COR_PLAYER)
               );
16 }

```

Listing 38: Função main - movimentacao da barra do jogador

Em seguida, o código percorre todas as bolas em jogo. O trecho a seguir mostra a movimentação de cada bola, de acordo com seu `Type_mov`, e o tratamento das colisões contra as paredes laterais e superior do tabuleiro:

```

1 bool *bola_morreu = (bool *)malloc(sizeof(bool) * num_bolas);
2 int bb = 0;
3 for (bb = 0; bb < num_bolas; bb++) bola_morreu[bb] = false;
4
5 int b = 0;
6 for (b = 0; b < num_bolas; b++) {
7     video_box(bolas[b].Pos_X, bolas[b].Pos_Y, bolas[b].Pos_X + 3,
8             bolas[b].Pos_Y + 3, resample_rgb(db, BACKGROUND_BLUE));
9
10    if (bolas[b].Type_mov == 0) bolas[b].Pos_Y += 1; //vertical pra cima
11    if (bolas[b].Type_mov == 1) bolas[b].Pos_Y -= 1; //vertical pra
        baixo
12    //4 diagonais
13    if (bolas[b].Type_mov == 2) { bolas[b].Pos_X -= 1; bolas[b].Pos_Y -=
        1; }
14    if (bolas[b].Type_mov == 3) { bolas[b].Pos_X += 1; bolas[b].Pos_Y -=
        1; }
15    if (bolas[b].Type_mov == 4) { bolas[b].Pos_X -= 1; bolas[b].Pos_Y +=
        1; }
16    if (bolas[b].Type_mov == 5) { bolas[b].Pos_X += 1; bolas[b].Pos_Y +=
        1; }
17    //rebate parede direita
18    if (bolas[b].Pos_X < 10) {
19        bolas[b].Pos_X = 10;
20        if (bolas[b].Type_mov == 2) bolas[b].Type_mov = 3; //caso
            subindo
21        if (bolas[b].Type_mov == 4) bolas[b].Type_mov = 5; //caso
            descendo
22    }
23    //rebate parede esquerda
24    if (bolas[b].Pos_X > 306) {
25        bolas[b].Pos_X = 306;
26        if (bolas[b].Type_mov == 3) bolas[b].Type_mov = 2; //caso
            subindo
27        if (bolas[b].Type_mov == 5) bolas[b].Type_mov = 4; //caso

```

```

28         }
29         if (bolas[b].Pos_Y < 10) {
30             bolas[b].Pos_Y = 10;
31             if (bolas[b].Type_mov == 1) bolas[b].Type_mov = 0;
32             if (bolas[b].Type_mov == 2) bolas[b].Type_mov = 4;
33             if (bolas[b].Type_mov == 3) bolas[b].Type_mov = 5;
34         }

```

Listing 39: Função main - movimento das bolas e colisao com paredes

Antes de mover, cada bola é apagada da posição antiga; a nova posição é calculada de acordo com o Type_mov atual, e em seguida são tratadas as colisões contra a parede esquerda, direita e superior do tabuleiro, invertendo a componente horizontal ou vertical do movimento conforme necessário. O vetor auxiliar bola_morreu, alocado no início do laço, marca quais bolas devem ser removidas ao final da iteração ao invés de removê-las imediatamente, o que bagunçaria os índices do laço. O trecho seguinte trata da queda da bola pela parte inferior da tela e da colisão contra a barra do jogador:

```

1  if (bolas[b].Pos_Y > 205) {
2      bola_morreu[b] = true;
3      continue;
4  }
5  //colisao com o player
6  if ((bolas[b].Pos_Y + 3 >= Y_PLAYER - 1) && (bolas[b].Pos_Y <= Y_PLAYER
7      + ALTURA_PLAYER - 1)) {
8      if ((bolas[b].Pos_X + 3 >= player_x) && (bolas[b].Pos_X <= player_x
9          + LARGURA_PLAYER)) {
10         if (bolas[b].Pos_X < player_x + 14) bolas[b].Type_mov = 2;
11         else if (bolas[b].Pos_X <= player_x + 26) bolas[b].Type_mov = 1;
12         else bolas[b].Type_mov = 3;
13     }
14 }

```

Listing 40: Função main - queda da bola e colisao com o player

Caso a bola ultrapasse a linha Pos_Y > 205, o que indica que ela ultrapassou o player e caiu do cenário. Logo, essa bola é marcada como morta e o restante do processamento daquela bola é ignorado. Em seguida, o código verifica a colisão da bola contra os blocos, mapeando a posição da bola diretamente para um índice da matriz blocos, sem a necessidade de percorrer a matriz inteira a cada quadro:

```

1  short teste_x = bolas[b].Pos_X;
2  short teste_y = bolas[b].Pos_Y;
3  //caso colisao blocos
4  if (bolas[b].Type_mov == 0) { teste_x = bolas[b].Pos_X + 1; teste_y =
5      bolas[b].Pos_Y + 3; }
6  if (bolas[b].Type_mov == 1) { teste_x = bolas[b].Pos_X + 1; teste_y =
7      bolas[b].Pos_Y; }

```

```

6  if (bolas[b].Type_mov == 2) { teste_x = bolas[b].Pos_X;      teste_y =
    bolas[b].Pos_Y;      }
7  if (bolas[b].Type_mov == 3) { teste_x = bolas[b].Pos_X + 3; teste_y =
    bolas[b].Pos_Y;      }
8  if (bolas[b].Type_mov == 4) { teste_x = bolas[b].Pos_X;      teste_y =
    bolas[b].Pos_Y + 3; }
9  if (bolas[b].Type_mov == 5) { teste_x = bolas[b].Pos_X + 3; teste_y =
    bolas[b].Pos_Y + 3; }
10
11 int col_bloco = (teste_x - 10) / 30;
12 int row_bloco = (teste_y - 10) / 11;

```

Listing 41: Função main - mapeamento da bola para o bloco correspondente

O ponto de teste (teste_x, teste_y) representa as coordenadas da bola. Dividindo esse ponto pela largura e altura aproximadas de um bloco (mais o espaçamento de 1 pixel entre eles), obtém-se diretamente os índices row_bloco e col_bloco da matriz blocos que a bola pode estar tocando, sem necessidade de nenhum laço de busca. O trecho a seguir verifica se esse bloco está de fato vivo e próximo o suficiente da bola, e trata sua destruição:

```

1  if ((row_bloco >= 0) && (row_bloco < 5) && (col_bloco >= 0) && (
    col_bloco < 10)) {
2  //verifica se esta perto o suficiente do bloco
3      bool perto_o_suficiente =
4          (bolas[b].Pos_X + 3 >= blocos[row_bloco][col_bloco].Pos_X) &&
5          (bolas[b].Pos_X      <= blocos[row_bloco][col_bloco].Pos_X +
            LARGURA_BLOCO - 1) &&
6          (bolas[b].Pos_Y + 3 >= blocos[row_bloco][col_bloco].Pos_Y) &&
7          (bolas[b].Pos_Y      <= blocos[row_bloco][col_bloco].Pos_Y +
            ALTURA_BLOCO - 1);
8
9      if (perto_o_suficiente && blocos[row_bloco][col_bloco].IsAlive) {
10         blocos[row_bloco][col_bloco].IsAlive = false;
11         video_box(blocos[row_bloco][col_bloco].Pos_X, blocos[row_bloco][
            col_bloco].Pos_Y,
12                 blocos[row_bloco][col_bloco].Pos_X + LARGURA_BLOCO -
                    1,
13                 blocos[row_bloco][col_bloco].Pos_Y + ALTURA_BLOCO - 1,
                    background_color);
14         score += 10;
15         AtualizarScore();

```

Listing 42: Função main - destruicao do bloco e liberacao de poder

A verificação perto_o_suficiente existe porque o mapeamento matemático direto de teste_x/teste_y para um índice da matriz é apenas uma aproximação, tendo em vista o espaçamento existe de 1 pixel entre eles. Essa segunda checagem, confirma que a bola de fato está encostando no bloco antes de destruí-lo, evitando a falsa

colisão no espaçamento de 1 pixel. Uma vez confirmada a colisão, o bloco é marcado como destruído, apagado da tela e a pontuação é atualizada. Em seguida, caso o bloco destruído seja de um dos quatro tipos especiais, o *power-up* correspondente é liberado com `adicionar_poder`, e por fim a bola é rebatida, invertendo seu `Type_mov` para o sentido oposto verticalmente:

```

1 //Verifica se deve liberar algum poder
2     if (blocos[row_bloco][col_bloco].Type ==
        BLOCO_TIPO_DUPLICAR_BOLA)
3         adicionar_poder(blocos[row_bloco][col_bloco].Pos_X +
        LARGURA_BLOCO / 2,
4                         blocos[row_bloco][col_bloco].Pos_Y, 0);
5     if (blocos[row_bloco][col_bloco].Type ==
        BLOCO_TIPO_AUMENTAR_PLAYER)
6         adicionar_poder(blocos[row_bloco][col_bloco].Pos_X +
        LARGURA_BLOCO / 2,
7                         blocos[row_bloco][col_bloco].Pos_Y, 1);
8     if (blocos[row_bloco][col_bloco].Type ==
        BLOCO_TIPO_DIMINUIR_PLAYER)
9         adicionar_poder(blocos[row_bloco][col_bloco].Pos_X +
        LARGURA_BLOCO / 2,
10                        blocos[row_bloco][col_bloco].Pos_Y, 2);
11    if (blocos[row_bloco][col_bloco].Type == BLOCO_TIPO_RELOGIO)
12        adicionar_poder(blocos[row_bloco][col_bloco].Pos_X +
        LARGURA_BLOCO / 2,
13                        blocos[row_bloco][col_bloco].Pos_Y, 3);
14
15 //Bola rebatida
16     if (bolas[b].Type_mov == 0) bolas[b].Type_mov = 1;
17     else if (bolas[b].Type_mov == 1) bolas[b].Type_mov = 0;
18     else if (bolas[b].Type_mov == 2) bolas[b].Type_mov = 4;
19     else if (bolas[b].Type_mov == 3) bolas[b].Type_mov = 5;
20     else if (bolas[b].Type_mov == 4) bolas[b].Type_mov = 2;
21     else if (bolas[b].Type_mov == 5) bolas[b].Type_mov = 3;
22 }
23 }
24
25 video_box(bolas[b].Pos_X, bolas[b].Pos_Y, bolas[b].Pos_X + 3,
26           bolas[b].Pos_Y + 3, resample_rgb(db, COR_PLAYER));
27 }

```

Listing 43: Função main - liberacao do poder e rebatimento da bola

Ao final do laço de bolas, cada uma é redesenhada em sua nova posição. Encerrado o processamento de todas as bolas, o trecho a seguir compacta o array `bolas`, removendo de uma só vez todas as que foram marcadas como mortas naquela iteração, e verifica se a lista ficou vazia, encerrando a partida com derrota nesse caso:

```

1 int write_idx = 0, read_idx = 0;

```

```

2  for (read_idx = 0; read_idx < num_bolas; read_idx++) {
3      if (!bola_morreu[read_idx]) {
4          if (write_idx != read_idx) bolas[write_idx] = bolas[read_idx];
5          write_idx++;
6      }
7  }
8  num_bolas = write_idx;
9  free(bola_morreu);
10
11 if (num_bolas == 0) {
12     free(bolas);
13     bolas = NULL;
14     estado = ESTADO_GAMEOVER;
15     fase_ativa = false;
16 }
17 if (!fase_ativa) break;

```

Listing 44: Função main - compactacao do array de bolas e verificacao de derrota

O trecho final do laço da fase 1 atualiza os *power-ups* em queda, controla a duração dos efeitos de largura da barra, verifica a transição para a fase 2, e aplica o *delay* entre um quadro e outro:

```

1  atualizar_poderes(background_color);
2  if (contador_expansao > 0) {
3      contador_expansao--;
4      if (contador_expansao == 0) {
5          video_box(player_x, Y_PLAYER, player_x + LARGURA_PLAYER - 1,
6                  Y_PLAYER + ALTURA_PLAYER - 1, resample_rgb(db,
7                      BACKGROUND_BLUE));
8          LARGURA_PLAYER = LARGURA_PLAYER_BASE;
9          tipo_efeito_player = 0;
10         if (player_x > 310 - LARGURA_PLAYER) player_x = 310 -
11             LARGURA_PLAYER;
12         video_box(player_x, Y_PLAYER, player_x + LARGURA_PLAYER - 1,
13                 Y_PLAYER + ALTURA_PLAYER - 1, resample_rgb(db,
14                     COR_PLAYER));
15     }
16 }
17
18 //verifica se destruiu todos os blocos da fase 1
19 if (score >= SCORE_ALVO_LEVEL1) {
20     estado = ESTADO_LEVEL2;
21     fase_ativa = false;
22     break;
23 }
24
25 //contador de prazo dos efeitos dos poderes
26 if (contador_relogio > 0) contador_relogio--;
27 //Deixa o tempo muito mais lento
28 if (contador_relogio > 0) delay(50000 * MULTIPLICADOR_DELAY_RELOGIO);

```

```
25 else delay(40000);
```

Listing 45: Função main

Ao expirar o contador de expansão/diminuição, a barra volta ao seu tamanho padrão e/ou ao delay normal. Assim que a pontuação atinge o alvo de 500 pontos, o jogo troca para ESTADO_LEVEL2, encerrando o laço da fase 1. Por fim, o *delay* entre quadros — que controla a velocidade geral do jogo — é multiplicado pelo fator MULTIPLICADOR_DELAY_RELOGIO enquanto o efeito de relógio estiver ativo, tornando o jogo perceptivelmente mais lento durante sua duração.

O estado ESTADO_LEVEL2 segue exatamente a mesma estrutura do estado ESTADO_LEVEL1 descrita anteriormente, ou seja, com movimentação da barra, movimento e colisão das bolas contra paredes, barra e blocos, atualização de *power-ups* e controle do *delay*, reaproveitando a mesma lógica de código, já que a mecânica do jogo não muda entre as fases, apenas o layout dos blocos e a condição de encerramento. A única diferença relevante está na preparação da fase e na condição de vitória, mostradas a seguir:

```
1 case ESTADO_LEVEL2: {
2     video_text(5, 55, text_score);
3     video_text(20, 55, text_pontoacao);
4     video_box(0, 0, STANDARD_X - 1, STANDARD_Y - 1, 0);
5     video_box(10, 10, 310 - 1, 210 - 1, background_color);
6
7     // Mantem o score acumulado da fase 1, so reseta bolas/poderes/
       player
8     int score_ao_entrar = score;
9     Reset(ESTADO_LEVEL2);
10    score = score_ao_entrar;
11    AtualizarScore();
12
13    bool fase_ativa = true;
14    while (fase_ativa) {
15        // ... movimentacao da barra, das bolas, colisoes e destruicao
           de blocos
16        // identicas ao estado ESTADO_LEVEL1 ...
17
18        if (score >= score_ao_entrar + SCORE_ALVO_LEVEL2) {
19            estado = ESTADO_YOUWIN;
20            fase_ativa = false;
21            break;
22        }
23    }
24    break;
25 }
```

Listing 46: Função main - preparacao e condicao de vitoria do estado LEVEL2

Como a função `Reset` sempre zera a pontuação, a pontuação acumulada na fase 1 é salva na variável local `score_ao_entrar` antes da chamada, e restaurada logo em seguida, garantindo que a pontuação exibida ao jogador continue de onde parou. A condição de vitória final é então verificada em relação a esse valor salvo, exigindo mais 300 pontos a partir do que já havia sido conquistado. Vale destacar que essa quantidade se refere à pontuação correspondente à destruição de todos os blocos da fase 2. Por fim, os dois últimos estados da máquina, `ESTADO_GAMEOVER` e `ESTADO_YOUWIN` consistem em desenhar a tela correspondente e aguardar o jogador pressionar `KEY2` para reiniciar:

```
1 case ESTADO_GAMEOVER: {
2     draw_background(0, 0, gameover, 320, 240); //tela perdeu
3     LimparTextoTela(); //limpar texto residual
4     while (1) {
5         int botoes = *KEY_ptr;
6         if (botoes & 0x04) { //reseta com botao
7             aguardar_soltar_tecla(KEY_ptr, 0x04);
8             LimparTextoTela();
9             estado = ESTADO_MENU;
10            break;
11        }
12    }
13    break;
14 }
15
16 case ESTADO_YOUWIN: {
17     draw_background(0, 0, ganhou, 320, 240); //tela ganhou
18     LimparTextoTela(); //limpar texto residual
19     while (1) {
20         int botoes = *KEY_ptr;
21         if (botoes & 0x04) { //reseta com botao
22             aguardar_soltar_tecla(KEY_ptr, 0x04);
23             LimparTextoTela();
24             estado = ESTADO_MENU;
25             break;
26         }
27     }
28    break;
29 }
30
31 }
32 return 0;
33 }
```

Listing 47: Função `main` - estados `GAMEOVER` e `YOUWIN`

Nos dois casos, a tela cheia correspondente (*sprite* `gameover` ou `ganhou`) é desenhada com `draw_background`, o texto residual é limpo, e o código aguarda ati-

vamente o pressionamento de KEY2, chamando `aguardar_soltar_tecla` antes de retornar ao `ESTADO_MENU`, reiniciando todo o ciclo do jogo a partir da tela inicial.

5 Execução do Projeto

Com todo o código do jogo desenvolvido e explicado na seção anterior, a execução na placa DE10-Standard segue exatamente o mesmo passo a passo já demonstrado na Figura 19 (na seção Primeira execução). Ao criar o projeto no *Intel FPGA Monitor Program*, selecione novamente *ARM Cortex-A9* como arquitetura e *DE10-Standard Computer* como sistema. Na etapa de *Program Type*, há duas possibilidades:

- Selecionar *C Program* e adicionar os arquivos `main.c`, `address_map_arm.h` e `sprites.h` do jogo na tela de gerenciamento de arquivos, seguindo o fluxo padrão já demonstrado. Nesse caso, após salvar o projeto, basta acessar o menu *Actions* e clicar em *Compile & Load* para compilar e carregar o jogo na placa;
- Caso o fluxo acima não funcione corretamente, utilizar o *script* `compilar.bat` para compilar o código externamente, conforme detalhado anteriormente, selecionando *AXF, ELF or SREC file* como *Program Type* e adicionando apenas o arquivo `main.srec` gerado. Nesse caso, após salvar o projeto, deve-se clicar apenas em *Load* no menu *Actions*.

Após o carregamento do programa ser concluído (seja via *Compile & Load* ou apenas *Load*), basta acessar novamente o menu *Actions* e clicar em *Continue*, ou simplesmente pressionar a tecla *F3*, para que o jogo seja executado de fato, com o resultado sendo exibido no monitor VGA conectado à placa.

Durante a execução, o jogo apresenta primeiramente a tela de menu inicial, aguardando que o jogador pressione o botão KEY0 para iniciar a partida. Durante o jogo, o botão KEY0 move a barra para a direita, o botão KEY1 move a barra para a esquerda, e o botão KEY2 reinicia a partida a qualquer momento, retornando ao menu. A Figura 33 apresenta a tela de menu inicial do jogo em execução na placa física, a Figura 34 apresenta o jogo em andamento na fase 1, e a Figura 35 apresenta a tela de vitória.

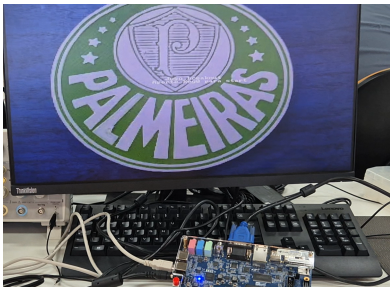


Figura 33: Tela de menu inicial do jogo

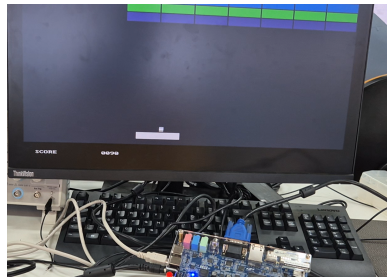


Figura 34: Jogo em execução na placa DE10-Standard

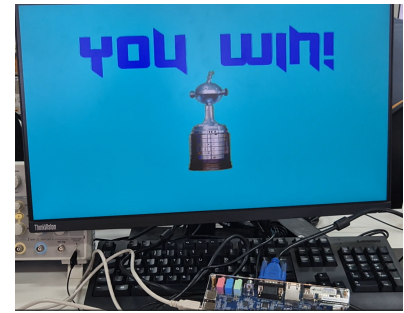


Figura 35: Tela de vitória

6 Conclusão

Ao longo deste tutorial, foi apresentada a base teórica necessária para a compreensão da arquitetura da placa Altera DE10-Standard e do processador ARM Cortex-A9, o passo a passo completo de instalação e configuração das ferramentas de desenvolvimento, e a explicação detalhada, função por função, de todo o código-fonte do jogo *Bricks*. Por fim, foi descrito o processo de execução do projeto na placa física.

Ao seguir todos os passos descritos neste documento, o leitor será capaz não apenas de compilar, carregar e executar o jogo aqui desenvolvido, mas também de compreender algum dos mecanismos de programação em um processador ARM em uma FPGA, habilitando-o a desenvolver, a partir dos mesmos conceitos e da mesma estrutura de código, outros jogos do tipo *arcade* na placa Altera DE10-Standard.

Como resultado deste trabalho, o jogo *Bricks* foi implementado com sucesso, partindo do código de exemplo *Video* fornecido pelo *Intel FPGA Monitor Program* e sobre ele construindo toda a lógica de um *Breakout* completo: renderização gráfica via *pixel buffer* com *sprites* personalizados, controle da barra, múltiplas bolas, um sistema de *power-ups*, duas fases com *layouts* de blocos diferentes, sistema de pontuação e uma máquina de estados que contém menu inicial, as duas fases, tela de derrota e tela de vitória.

Fontes das Imagens Utilizadas

As imagens utilizadas neste projeto foram obtidas a partir de fontes públicas disponíveis na Internet. Quando foi possível, foi indicada a fonte original. No entanto, houve casos em que a página original não estava mais disponível, então foi indicada a pesquisa realizada no Google Imagens utilizada para localizar o material.

- Troféu da Copa Libertadores (tela de vitória):
Pesquisa no Google Imagens: <https://www.google.com/search?q=trofeu+libertadores>

- Imagem utilizada na tela de *Game Over*:
Pesquisa no Google Imagens: <https://www.google.com/search?q=imagens+320x240+bem+dark>
- Escudo da Sociedade Esportiva Palmeiras (imagem base para o menu):
<https://www.printables.com/model/1236267-escudo-palmeiras/related>

Referências

- [1] INTEL CORPORATION. *DE10-Standard Computer System with ARM Cortex A9*. FPGA University Program, maio 2020. Disponível em: https://fpgacademy.org/Downloads/DE10-Standard_Computer_ARM.pdf. Acesso em: 05 jul. 2026.
- [2] ALTERA CORPORATION. *Cyclone V SoC FPGA: ARM Cortex-A9 MPCore Processor — Advance Information Brief*. Disponível em: https://people.ece.cornell.edu/land/courses/ece5760/DE1_S0C/aib-01020-soc-fpga-cortex-a9-processor.pdf. Acesso em: 05 jul. 2026.
- [3] NASCIMENTO, Erick Nogueira do; PIETZ, Franz; WATANABE, Lucas. *Processador ARM Cortex-A9*. Instituto de Computação, Universidade Estadual de Campinas (Unicamp), disciplina MO401, 2012. Disponível em: https://www.ic.unicamp.br/~ducatte/mo401/1s2012/T2/G07-032483_076673_134068-t2.pdf. Acesso em: 05 jul. 2026.
- [4] HENNESSY, John L.; PATTERSON, David A. *Arquitetura de computadores: uma abordagem quantitativa*. 3. ed. Rio de Janeiro: Campus, 2003. 827 p. ISBN 85-352-1110-1.
- [5] FPGACADEMY. *FPGAcademy: Tools*. Disponível em: <https://fpgacademy.org/tools>. Acesso em: 05 jul. 2026.
- [6] ALTERA. *Quartus Prime Lite Edition Design Software Version 23.1 for Windows*. Disponível em: <https://www.altera.com/downloads/fpga-development-tools/quartus-prime-lite-edition-design-software-version-23-1-windows>. Acesso em: 05 jul. 2026.
- [7] PISKEL. *Piskel: Free Online Sprite Editor*. Disponível em: <https://www.piskelapp.com/p/create/sprite/>. Acesso em: 05 jul. 2026.
- [8] MICROSOFT. *Introdução ao Windows Subsystem for Linux*. Disponível em: <https://learn.microsoft.com/pt-br/windows/wsl/tutorials/linux>. Acesso em: 05 jul. 2026.